

Image Encoders with DINO and DINOv2

Generated by ChatGPT

1970-01-01

Contents

1	Introduction	9
1.1	A New Paradigm in Artificial Intelligence	9
1.2	Why World Models Matter	10
1.3	What This Book Covers	10
1.4	A Glimpse into the Future	11
2	Chapter 1: Introduction to World Models	13
2.1	1.1 What Are World Models?	13
2.2	1.2 The Evolution of World Models	14
2.3	1.3 Key Properties of World Models	15
2.4	1.4 Applications of World Models	16
2.5	1.5 Future Directions in World Models	16
2.6	1.6 Conclusion	17
3	Introduction	19
4	Self-Attention Mechanism	21
4.1	Mathematical Formulation	21
4.2	Multi-Head Attention	21
5	Transformer Encoder Architecture	23
5.1	Transformer Architectures: BERT vs. GPT vs. Encoder-Decoder Models	24
5.2	Masked Language Modeling (MLM)	25

5.3	Encoder Decoder Models (e.g. BERT)	25
5.4	Summary of key of transformer Architecture	27
5.5	Model Comparison	27
6	Vision Transformers (ViTs)	29
6.1	Patch Extraction and Encoding	29
7	Contrastive Language-Image Pretraining (CLIP)	31
7.1	Architecture	31
7.2	Training Objective	31
7.3	Applications of CLIP	32
8	DINO: Self-Supervised Vision Encoding	33
8.1	Key Features of DINO	33
8.2	DINO Training Process	33
9	DINOv2: Improved Vision Encoding	37
9.1	Key Improvements in DINOv2	37
9.2	Masked Image Modeling (MIM)	37
9.3	Training Objectives	38
9.4	Applications of DINO	38
10	GENERATIVE MODELS	41
11	GENERATIVE MODEL	43
12	Introduction to GAN Models	45
12.1	GAN Architecture	45
12.2	Training GANs	46
12.3	Key GAN Model Architectures	47
12.4	1. Vanilla GAN	47
12.5	2. Deep Convolutional GAN (DCGAN)	47
12.6	3. Conditional GAN (CGAN)	48
12.7	4. Wasserstein GAN (WGAN)	48

<i>CONTENTS</i>	5
12.8 5. CycleGAN	49
12.9 6. Progressive GAN (ProGAN)	49
12.10 7. StyleGAN	50
12.11 Summary	50
12.12 Variational Autoencoders (VAEs)	51
13 2. Kullback-Leibler (KL) Divergence	53
14 3. Derivation of the VAE Loss Function	55
15 4. Variational Autoencoder (VAE) Architecture	57
15.1 Summary	58
16 Why the Field Moved Away from VAEs and GANs	59
16.1 1. Issues with GANs (Generative Adversarial Networks)	59
16.2 2. Issues with VAEs (Variational Autoencoders)	60
16.3 3. Shift Towards New Models	61
16.4 4. Why the Shift?	62
16.5 Conclusion	63
17 DIFFUSION MODELS	65
17.1 1. Introduction to DDPMs	65
17.2 2. Mathematical Formulation of Diffusion Models	65
17.3 2.1 Forward Diffusion Process	66
17.4 3. Reverse Diffusion Process	66
17.5 4. Derivation of the DDPM Loss Function Using KL Divergence	67
17.6 IMPLEMENTATION OF DDPM	68
17.7 Learning the Reverse Process in Diffusion Models	69
17.8 UNETS	70
18 1. Introduction to Score-Based Generative Models (SGMs)	71
19 2. Mathematical Formulation of Score-Based Models	73
20 3. Reverse Process: Score-Based Generative Model	75

21	4. Derivation of the Score-Based Loss Function Using KL Divergence	77
22	Chapter: Reinforcement Learning - Theory and Applications	81
22.1	1. Introduction	81
22.2	2. Fundamentals of Reinforcement Learning	81
22.3	3. Types of Reinforcement Learning	82
22.4	4. Popular RL Algorithms	83
22.5	5. Applications of Reinforcement Learning	83
22.6	6. Challenges in RL	84
22.7	7. Conclusion and Future Directions	84
23	Markov Decision Process (MDP)	85
23.1	State Transition Probability	86
23.2	Policy in MDP	86
23.3	Optimal Policy (π^*)	86
23.4	Bellman Equation	87
23.5	State-Value Function ($V(s)$)	87
23.6	Action-Value Function ($Q(s, a)$)	87
23.7	Optimal Policy & Bellman Optimality Equation	88
23.8	Conclusion	88
23.9	Reinforcement learning	88
24	Reward-Based Learning	91
24.1	Types of Rewards	91
24.2	Reward Function	91
24.3	Optimizing Rewards	92
24.4	Exploration vs. Exploitation	92
25	Model-Based Reinforcement Learning	93
25.1	Monte Carlo Tree Search (MCTS)	93

<i>CONTENTS</i>	7
26 Model-Free Reinforcement Learning	97
26.1 Value-Based Learning	98
26.2 Q-Learning	98
26.3 Deep Q-Network (DQN)	100
26.4 Applications of DQN	101
26.5 Policy Gradient Methods	102
26.6 Types of Policy Gradient Methods	103
26.7 REINFORCE (Monte Carlo Policy Gradient): Uses episode returns to update policies.	103
26.8 REINFORCE Algorithm Steps	104
26.9 Proximal Policy Optimization (PPO)	105
26.10 The Advantage Function	105
26.11 PPO Algorithm Steps	106
26.12 Comparison with REINFORCE	106
26.13 Applications of PPO	107
26.14 Actor-Critic Methods	107
26.15 Types of Actor-Critic Methods	109
27 Conclusion	111
28 Introduction	113
29 Self-Attention Mechanism	115
29.1 Mathematical Formulation	115
29.2 Multi-Head Attention	115
30 Transformer Encoder Architecture	117
30.1 Masked Language Modeling (MLM)	118
31 Vision Transformers (ViTs)	119
31.1 Patch Extraction and Encoding	119

32 Contrastive Language-Image Pretraining (CLIP)	121
32.1 Architecture	121
32.2 Training Objective	121
32.3 Applications of CLIP	122
33 DINO: Self-Supervised Vision Encoding	123
33.1 Key Features of DINO	123
33.2 DINO Training Process	123
34 DINOv2: Improved Vision Encoding	125
34.1 Key Improvements in DINOv2	125
34.2 Masked Image Modeling (MIM)	125
34.3 Training Objectives	126
35 Conclusion	127

Chapter 1

Introduction

“All models are wrong, but some are useful.”
— George E. P. Box

1.1 A New Paradigm in Artificial Intelligence

Artificial Intelligence has undergone remarkable transformations in the past decade, from mastering complex games like **Go** (Silver et al., 2016) to generating **realistic images and text** (Radford et al., 2021; Ramesh et al., 2022). However, many AI models today remain **reactive** rather than **truly intelligent**—they do not plan, anticipate, or imagine possible futures.

This is where **World Models** come into play. Inspired by human cognition, World Models provide AI systems with an **internal representation of the environment**, enabling them to **simulate, predict, and act efficiently**. Instead of relying on vast amounts of trial-and-error learning, World Models allow machines to **imagine outcomes before taking action**.

The foundations of **World Models** lie at the intersection of:

- **Latent Variable Models: Variational Autoencoders (VAEs)** (Kingma & Welling, 2013) and **Normalizing Flows** (Rezende & Mohamed, 2015) allow AI to learn compact representations of the world.
- **Reinforcement Learning (RL) and Model-Based Planning:** **Dyna-Q** (Sutton, 1991) and **MuZero** (Schrittwieser et al., 2019) demonstrate how AI can plan using learned models.
- **Generative Modeling: Generative Adversarial Networks (GANs)** (Goodfellow et al., 2014) and **Denoising Diffusion Probabilistic**

Models (DDPMs) (Ho et al., 2020) improve AI’s ability to generate realistic predictions.

- **Neuroscience-Inspired AI: Predictive Coding** (Friston, 2005) suggests that intelligence emerges from minimizing prediction errors—an idea deeply aligned with World Models.

The breakthrough work of **David Ha and Schmidhuber (2018)** on learning **recurrent world models** demonstrated how AI could self-learn world representations **without explicit supervision**, enabling agents to **simulate entire environments** in a latent space before interacting with the real world.

1.2 Why World Models Matter

Unlike traditional AI models that rely on brute-force data processing, World Models focus on **structured learning, generalization, and imagination**. This shift has profound implications for:

- **Autonomous Systems:** Self-driving cars (**Waymo, Tesla FSD**) use predictive world models for decision-making.
- **Reinforcement Learning:** Instead of trial-and-error, agents like **DreamerV3** (Hafner et al., 2023) use world models to imagine future actions efficiently.
- **Creative AI:** Diffusion models like **DALL·E 3** (Ramesh et al., 2023) leverage latent world models for **generative creativity**.
- **Scientific Discovery:** AI-driven simulations (e.g., **AlphaFold** (Jumper et al., 2021)) revolutionize fields like **biology and physics**.

1.3 What This Book Covers

This book is a deep dive into the **theory, architectures, and applications** of World Models. The journey begins with **historical foundations**, progresses through **modern architectures**, and culminates in **real-world applications and future challenges**.

- **Part I: Foundations** – The evolution of world models from neuroscience, cognitive science, and reinforcement learning.

- **Part II: Architectures** – Deep learning models for learning world representations, including VAEs, transformers, and diffusion models.
- **Part III: Applications** – Case studies in robotics, game AI, and creative generative models.
- **Part IV: Future Directions** – How world models shape the path toward Artificial General Intelligence (AGI).

1.4 A Glimpse into the Future

The ability of AI to **develop internal representations of the world** is a stepping stone toward **artificial general intelligence (AGI)**. As AI systems become **more predictive, imaginative, and autonomous**, they will reshape **robotics, scientific discovery, and human-AI collaboration**.

This book serves as an invitation to explore one of the most exciting frontiers in AI. Whether you are a **researcher, engineer, or enthusiast**, understanding World Models is key to unlocking the next generation of AI.

Let's embark on this journey together.

Chapter 2

Chapter 1: Introduction to World Models

2.1 1.1 What Are World Models?

World models are AI architectures designed to construct internal representations of the environment, facilitating reasoning, prediction, and decision-making. They enable AI agents to simulate future scenarios, make informed decisions, and adapt dynamically to complex environments. These models are inspired by cognitive science, neuroscience, and reinforcement learning, playing a critical role in applications like robotics, game AI, autonomous vehicles, and scientific modeling.

2.1.1 1.1.1 The Concept of Internal Representations

The essence of world models lies in their ability to learn structured representations of an environment. By encoding sensory data into an internal model, an AI system can predict future states, evaluate different courses of action, and optimize long-term outcomes. This mirrors the way human cognition operates—we constantly build and refine mental models to navigate the world effectively.

Key aspects of internal representations include: - **Spatial Representations:** Understanding physical layouts and object relationships. - **Temporal Dynamics:** Predicting how states evolve over time. - **Causal Inference:** Learning cause-and-effect relationships. - **Uncertainty Estimation:** Handling incomplete or noisy data.

2.1.2 1.1.2 The Role of World Models in AI

World models bridge the gap between raw perception and intelligent behavior. Instead of directly mapping inputs to outputs (as in model-free reinforcement learning), these models construct intermediate representations, allowing agents to generalize knowledge across different tasks and environments. This ability is fundamental to achieving human-level intelligence.

2.2 1.2 The Evolution of World Models

The concept of world models has evolved through several key phases in AI history:

2.2.1 1.2.1 Early Symbolic AI (1950s-1980s)

- Early AI systems relied on rule-based representations (e.g., expert systems) to encode structured knowledge about the world.
- Limitations: These systems were brittle and failed to adapt to new, unseen environments.

2.2.2 1.2.2 Neural Networks & Connectionism (1980s-2000s)

- The rise of artificial neural networks enabled data-driven learning, reducing reliance on manually defined rules.
- However, these models lacked explicit world representations and struggled with long-term planning.

2.2.3 1.2.3 Deep Learning & Generative Models (2010s-Present)

- Advances in deep generative models, such as Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs), allowed AI to learn probabilistic representations of complex environments.
- These models demonstrated the ability to synthesize realistic data but lacked temporal coherence for sequential decision-making.

2.2.4 1.2.4 Model-Based Reinforcement Learning (2019-Present)

- Model-based RL frameworks, such as MuZero and DreamerV3, showcase the power of learning internal world models for decision-making.
 - These systems leverage learned representations to simulate future trajectories and optimize actions efficiently.
-

2.3 1.3 Key Properties of World Models

A robust world model should exhibit the following properties:

2.3.1 1.3.1 Generalization

- The ability to apply learned representations to new, unseen environments.
- Ensures AI does not overfit to specific training conditions.

2.3.2 1.3.2 Temporal Consistency

- Predicts how the environment evolves over time.
- Essential for planning and long-term decision-making.

2.3.3 1.3.3 Scalability

- Capable of handling increasingly complex environments.
- Allows adaptation from simple simulations to real-world applications.

2.3.4 1.3.4 Robustness

- Resilient to noise, uncertainty, and missing information.
- Ensures stability in dynamic and unpredictable environments.

2.3.5 1.3.5 Efficiency

- Optimizes decision-making while minimizing computational costs.
 - Balances model complexity with real-time responsiveness.
-

2.4 1.4 Applications of World Models

The integration of world models into AI systems has revolutionized various fields:

2.4.1 1.4.1 Robotics

- Enables autonomous robots to navigate, interact, and learn from their environments.
- Examples: Boston Dynamics' robots, robotic arms in manufacturing.

2.4.2 1.4.2 Gaming & Simulation

- AI agents utilize world models to plan strategies and adapt to new scenarios.
- Examples: AlphaZero, OpenAI's Dota 2 agent.

2.4.3 1.4.3 Autonomous Vehicles

- Predicts traffic patterns, obstacles, and potential hazards.
- Example: Tesla's Full Self-Driving (FSD) system.

2.4.4 1.4.4 Natural Language Processing (NLP)

- Enhances contextual understanding and coherence in text generation.
- Example: Large Language Models like GPT-4.

2.4.5 1.4.5 Healthcare & Biomedicine

- AI-powered simulations assist in drug discovery and disease modeling.
- Example: DeepMind's AlphaFold for protein structure prediction.

2.5 1.5 Future Directions in World Models

The field of world models is rapidly evolving, with research focused on:

2.5.1 1.5.1 Hybrid AI Architectures

- Combining deep learning with symbolic reasoning to improve interpretability.

2.5.2 1.5.2 Multimodal Learning

- Integrating vision, language, and sensory data for richer world representations.

2.5.3 1.5.3 Cognitive AI

- Designing AI that mimics human reasoning and problem-solving capabilities.

2.5.4 1.5.4 Continual Learning

- Enabling AI agents to learn and adapt over time without catastrophic forgetting.

2.5.5 1.5.5 Towards General AI

- Developing AI systems that generalize across domains and achieve human-level intelligence.

2.6 1.6 Conclusion

World models represent a transformative shift in AI research, enabling intelligent systems to learn structured representations, predict outcomes, and make informed decisions. From model-based reinforcement learning to generative modeling, these frameworks pave the way toward more autonomous, efficient, and adaptable AI systems. Future advancements in hybrid architectures, cognitive AI, and continual learning will further refine and expand the capabilities of world models, bringing us closer to achieving artificial general intelligence (AGI).

Key Takeaways: - World models build internal representations of the environment, facilitating decision-making and adaptability. - Their evolution spans from rule-based AI to deep learning and model-based reinforcement learning.

- Key properties include generalization, temporal consistency, robustness, and efficiency.
- Applications range from robotics and gaming to NLP and health-care.
- Future research focuses on hybrid architectures, multimodal learning, and cognitive AI.

The next chapters will delve into the mathematical foundations, probabilistic models, and deep learning frameworks that underpin world models, setting the stage for advanced AI research and applications.

Chapter 3

Introduction

Image encoders have evolved significantly with the emergence of **self-supervised learning** techniques. Recent advancements like **DINO** and **DINOv2** have demonstrated strong performance in learning feature representations without labeled data. In this post, we explore self-attention, transformers, and how they lead to advanced vision encoders.

Chapter 4

Self-Attention Mechanism

Self-attention is a key component of transformer architectures and is responsible for capturing **long-range dependencies** in input sequences. It enables the model to assign different importance weights to various parts of the input, allowing for more dynamic feature extraction.

4.1 Mathematical Formulation

Given an input sequence of token embeddings X , we compute **Query (Q)**, **Key (K)**, and **Value (V)** matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where W_Q , W_K , and W_V are learnable weight matrices. These matrices help in computing attention scores and refining feature representations.

The **scaled dot-product attention** is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where d_k is the dimension of the key vectors, which stabilizes gradient updates.

4.2 Multi-Head Attention

Instead of computing a single attention function, transformers use **multi-head attention**, which allows the model to focus on different parts of the sequence simultaneously. The attention heads are computed as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W_O$$

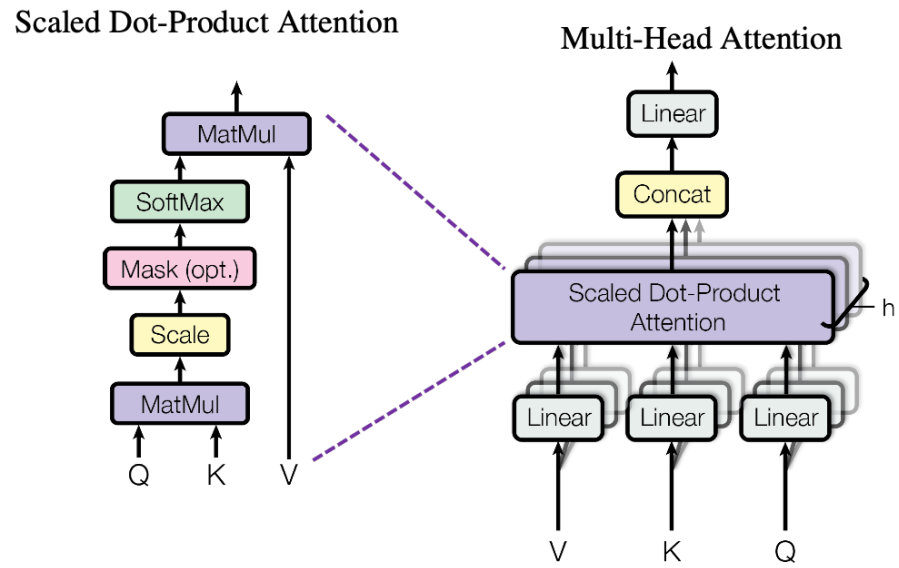


Figure 4.1: Attention

where each attention head computes attention independently before being concatenated and linearly transformed.

Chapter 5

Transformer Encoder Architecture

The **Transformer encoder** consists of multiple stacked layers, each containing:

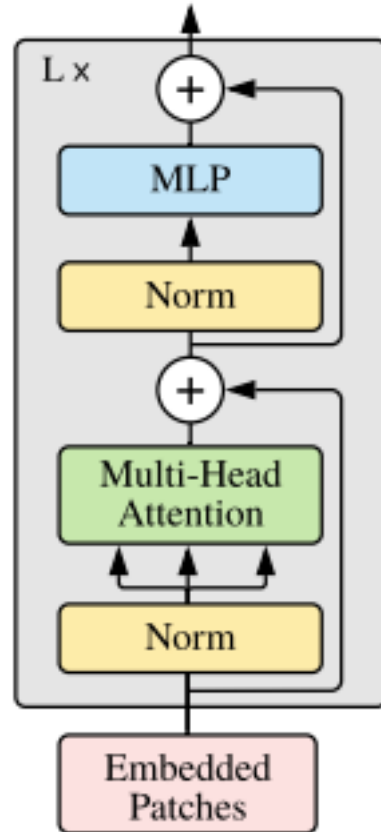
- Multi-head self-attention mechanism
- Feed-forward network (FFN)
- Residual connections and Layer Normalization

Each encoder block follows this sequence:

1. Input embeddings + Positional Encoding
2. Multi-head self-attention
3. Layer Normalization
4. Feed-forward network
5. Layer Normalization (again)

The encoder processes the input in parallel, unlike RNNs, making it more efficient for long sequences.

Transformer Encoder



Input to Transformers

There transformer models consist of the following inputs

1. Token Embedding generated using word2vec or similar word embeddings
2. Segment Embedding
3. Position Embedding

5.1 Transformer Architectures: BERT vs. GPT vs. Encoder-Decoder Models

Transformer models revolutionized deep learning by replacing RNNs and CNNs in NLP tasks. The core difference between BERT, GPT, and Encoder-Decoder models lies in how they use the Transformer components: encoder, decoder, or both.

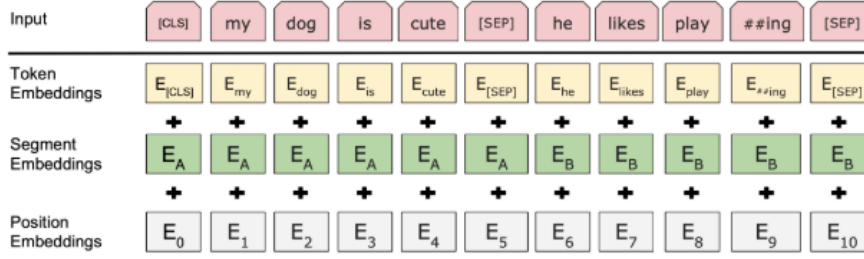


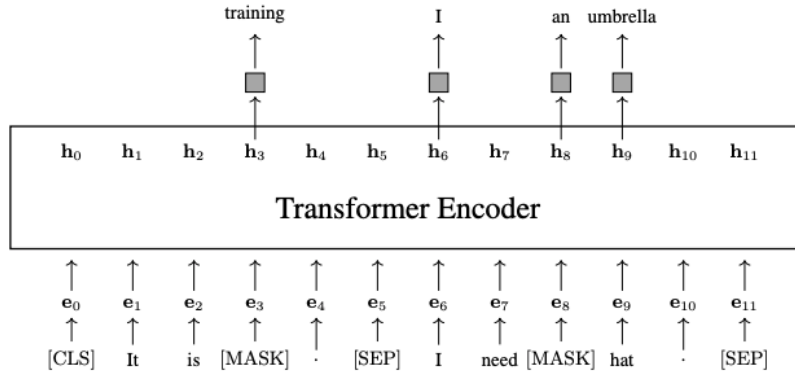
Figure 5.1: Input to transformer

5.2 Masked Language Modeling (MLM)

MLM is commonly used in natural language processing (NLP) tasks, where a fraction of the input tokens are randomly masked, and the model learns to predict the missing tokens based on their context. The objective function for MLM is:

$$\mathcal{L}_{MLM} = - \sum_{i \in M} \log P(x_i | x_{\setminus i})$$

where M represents the set of masked positions, and $P(x_i | x_{\setminus i})$ is the probability of correctly predicting the masked token.



##Decoder Models (e.g. GPT)

GPT is a decoder model

5.3 Encoder Decoder Models (e.g. BERT)

BERT IS and example of encoder decoder model

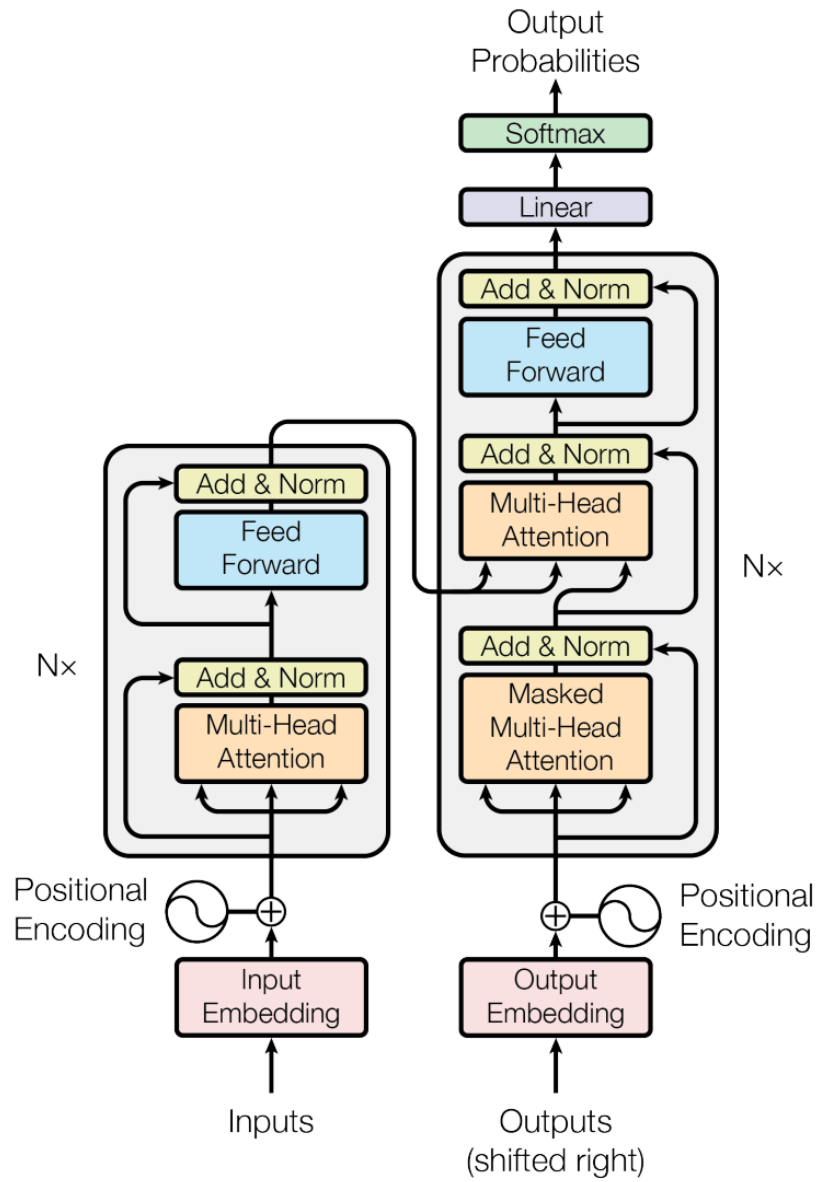


Figure 5.2: Transformer

5.4 Summary of key of transformer Architecture

5.5 Model Comparison

Feature	BERT (Encoder-Only)	GPT (Decoder-Only)	Encoder-Decoder (T5, BART, etc.)
Architecture	Encoder-only	Decoder-only	Both Encoder & Decoder
Directionality	Bidirectional	Unidirectional (left-to-right)	Bidirectional encoding, left-to-right decoding
Training Objective	MLM (Masked Language Model)	CLM (Causal Language Model)	Various (e.g., seq-to-seq, denoising)
Main Usage	Understanding	Generation	Translation, Summarization

Chapter 6

Vision Transformers (ViTs)

ViTs apply **self-attention** to image data, treating images as sequences of patches rather than using convolutional operations.

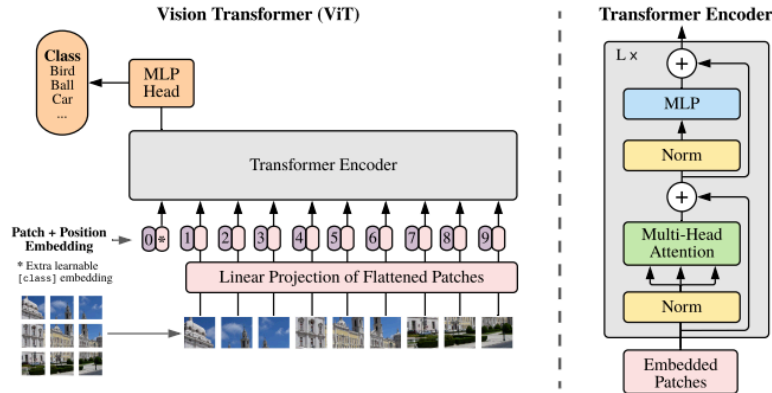


Figure 6.1: VISION TRANSFORMER

6.1 Patch Extraction and Encoding

An image is split into patches, flattened, and passed through a **linear projection**:

$$X_{patch} = W_E \cdot X_{input} + b_E$$

where W_E is the embedding matrix and b_E is a bias term. These embeddings are then processed using self-attention layers to extract contextual information.

Chapter 7

Contrastive Language-Image Pretraining (CLIP)

CLIP is a multimodal self-supervised learning model designed to learn joint representations of images and text. It is trained on a large dataset of image-text pairs using a contrastive learning approach.

7.1 Architecture

CLIP consists of two main components:

- An image encoder (ResNet or Vision Transformer)
- A text encoder (Transformer-based model)

Both encoders project images and text into a shared embedding space, where semantically similar images and text have higher similarity scores.

7.2 Training Objective

CLIP uses a contrastive learning objective where it maximizes the cosine similarity between matching image-text pairs while minimizing similarity between mismatched pairs:

$$\mathcal{L}_{CLIP} = - \sum_i \log \frac{e^{\cos(I_i, T_i)/\tau}}{\sum_j e^{\cos(I_i, T_j)/\tau}}$$

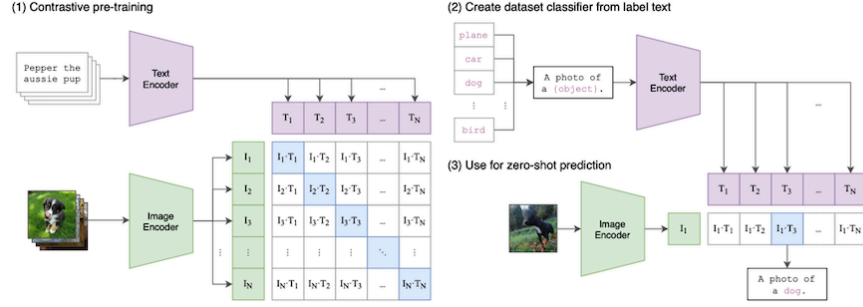


Figure 7.1: CLIP Model

where I_i and T_i are corresponding image-text pairs, and τ is a temperature parameter that controls the sharpness of the distribution.

7.3 Applications of CLIP

- Zero-shot image classification
- Image retrieval and search
- Generating text-based descriptions of images
- Few-shot learning for downstream vision tasks

CLIP's ability to understand images and text jointly makes it a powerful model for many vision-language applications.

Chapter 8

DINO: Self-Supervised Vision Encoding

DINO (Self-Distillation without Labels) is a **self-supervised learning method** that trains a **student** network to predict the output of a momentum **teacher** network.

8.1 Key Features of DINO

- Works with both CNNs and ViTs
- Learns representations using **contrastive learning**
- Uses a momentum teacher-student setup
- Multi-crop augmentation for efficient feature learning
- Achieves competitive performance in unsupervised representation learning

8.2 DINO Training Process

The DINO training process uses a self-supervised learning (SSL) approach that can be interpreted as a form of knowledge distillation without labels. It involves a student network and a teacher network that have the same architecture but different parameters. The key steps in the DINO training process are as follows:

- **Input and Augmentation:** An input image is passed through two different random transformations, creating two distorted views. These views are fed into the student and teacher networks.

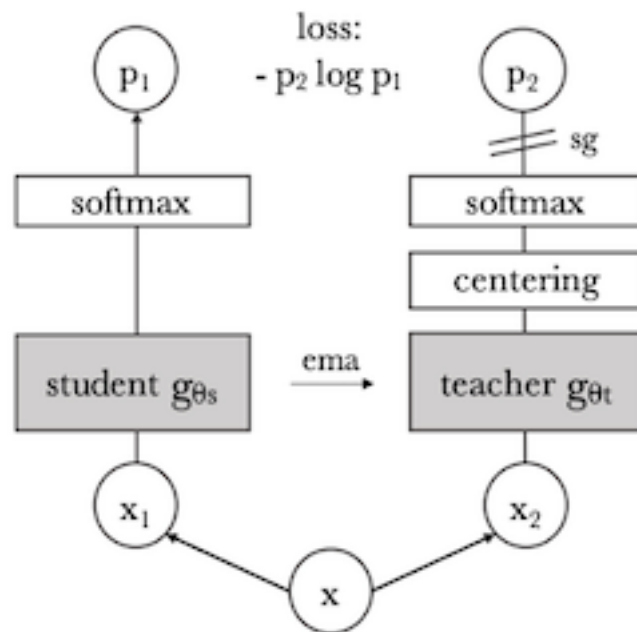


Figure 8.1: ENCODER

- **Student and Teacher Networks:** The student network (g_s) and the teacher network (g_t) process the augmented images, outputting feature maps. The teacher network's output is centered by subtracting the mean over the batch. Both the student's and teacher's outputs are normalized using a softmax function with a temperature parameter that controls the output distribution.

- **Loss Calculation:** A cross-entropy loss is calculated by measuring the similarity between the probability distributions from the student network and the teacher network. The student network is trained to match the output of the teacher network using this loss function. DINO minimizes the cross-entropy loss between teacher and student outputs:

$$\mathcal{L}_{DINO} = - \sum_i p_i^T \log p_i^S$$

where p_i^T and p_i^S are teacher and student predictions.

- **Teacher Update:** The teacher network's parameters are updated using an exponential moving average (EMA) of the student network's parameters. Gradient back-propagation is limited to the student network. The teacher network is updated by the formula: $c \leftarrow mc + (1-m) \frac{1}{B} \sum_{i=1}^B g_t(x_i)$, where 'm' is a rate parameter and 'B' is the batch size.

- **Centering and Sharpening:** The teacher network's output is centered with a mean computed over the batch, and output sharpening is achieved by using a low temperature value in the teacher's softmax normalisation.

- **Multi-Crop Training:** The student is fed multiple crops of an image, and it must align its local feature representations with those predicted by the teacher network for the global views of the image. This helps the network learn distinctive local features required for tasks like semantic segmentation.

- **Momentum Encoder:** The momentum teacher constantly outperforms the student during training. This is achieved by updating the teacher parameters using an exponential moving average of the student's parameters. The momentum encoder helps to stabilise training and improves performance.

- **Multi-crop Augmentation:** Training with multiple crops of an image improves performance. DINO benefits significantly from multi-crop training.

- **Cross-Entropy Loss:** Using a cross-entropy loss on the sharpened softmax outputs is an important component for good features.

- **Centering:** Centering the teacher output is important in avoiding collapse, particularly when a predictor or batch normalisation is not used.

The DINO framework is flexible and works on both convolutional neural networks (convnets) and Vision Transformers (ViTs) without modifying the architecture or internal normalisations

Chapter 9

DINOv2: Improved Vision Encoding

DINOv2 extends DINO with **Masked Image Modeling (MIM)** and **Self-Supervised Instance Discrimination**, improving its ability to capture both local and global features.

9.1 Key Improvements in DINOv2

- Introduces **masked image modeling (MIM)** to learn local textures
- Uses **multi-crop augmentation** to enhance representation learning
- No reliance on **text supervision** (unlike CLIP)
- Improved transferability to **dense prediction tasks** (e.g., segmentation)
- Asymmetric teacher-student design for more effective training
- Incorporates KoLeo regularization to improve feature uniformity

9.2 Masked Image Modeling (MIM)

MIM extends the concept of MLM to images by randomly masking patches in an image and training the model to reconstruct the missing information. The objective function for MIM is:

$$\mathcal{L}_{MIM} = - \sum_{i \in M} \log P(I_i | I_{\setminus i})$$

where I_i represents the masked image patch, and $I_{\setminus i}$ denotes the observed patches.

9.3 Training Objectives

DINOv2 uses a **hybrid objective**:

$$\mathcal{L}_{DINOv2} = \lambda_1 \mathcal{L}_{MIM} + \lambda_2 \mathcal{L}_{Instance}$$

where:

- $\mathcal{L}_{MIM}$ is the loss for **masked image modeling**, focusing on reconstructing masked patches
- $\mathcal{L}_{Instance}$ is the loss for **instance discrimination**, ensuring robust feature alignment across augmentations

DINOv2 leverages a **vision-only training approach**, making it highly effective for self-supervised learning in image-based tasks.

9.4 Applications of DINO

DINO and DINOv2 represent significant advances in **self-supervised learning for vision tasks**. Their ability to learn rich representations **without labeled data** makes them valuable for diverse applications such as **image classification, retrieval, segmentation, and biomedical imaging**.

Some applications of DINO include:

- **Image Classification:** DINO can be used for image classification tasks, achieving high accuracy on benchmarks such as ImageNet. It has been shown to outperform other self-supervised methods when used with ViTs. For example, DINO achieves 80.1% top-1 accuracy on ImageNet in linear evaluation with ViT-Base, which is superior to previous self-supervised features.
- **Feature Extraction:** DINO can extract robust visual features that are useful for various downstream tasks. These features capture semantic information about the image, and are effective when used in combination with k-NN classifiers. DINO features have demonstrated excellent k-NN classification performance. For example, a ViT-S/8 trained with DINO reached 78.3% top-1 accuracy on ImageNet with a k-NN classifier.
- **Object Discovery and Segmentation:** Self-supervised ViT features from DINO contain explicit information about the semantic segmentation of an image. This

is a unique property that is not as apparent with supervised ViTs or with convnets. DINO trained ViTs are able to separate objects.

- **Image Retrieval:** DINO features can be used for image retrieval tasks. DINO outperforms supervised learning methods in image retrieval tasks. When trained on the Google Landmarks v2 (GLDv2) dataset, DINO ViT features perform exceptionally well for landmark retrieval.

- **Copy Detection:** DINO can be applied to copy detection tasks where the goal is to identify images that have been distorted. ViTs trained with DINO are very competitive in this application.

- **Transfer Learning:** Features pre-trained with DINO transfer effectively to various downstream tasks, outperforming supervised pre-training in many cases. Fine-tuning DINO pre-trained models on downstream tasks leads to improved results compared to models trained with supervision. Self-supervised pre-training with DINO significantly improves results on ImageNet, achieving 1-2% improvement over supervised pre-training.

- **Low-shot learning:** DINO can be used for low-shot learning, where models are trained with very little labelled data. DINO has shown strong performance in low-shot learning scenarios.

- **Medical Imaging:** The DINO framework can be adapted for medical imaging applications. RAD-DINO, a model pre-trained using the DINOv2 framework, demonstrates that high-quality biomedical image encoders can be trained using only unimodal imaging data, without relying on text supervision. This is especially useful where there is a scarcity of paired image-text data. RAD-DINO performs comparably or better than state-of-the-art language-supervised models on image classification, semantic segmentation, and text report generation from images.

- **Image classification** RAD-DINO achieves similar or superior performance levels on image classification tasks compared to language-supervised models.

- **Semantic segmentation** RAD-DINO performs well on semantic segmentation tasks without needing large, densely annotated training datasets.

- **Report generation** RAD-DINO excels at generating accurate text reports from images, outperforming models trained with language supervision.

- **Prediction of demographic information** RAD-DINO can more accurately predict patient demographic information (e.g., sex or age) compared to language-supervised models.

Chapter 10

GENERATIVE MODELS

Chapter 11

GENERATIVE MODEL

##GAN

Chapter 12

Introduction to GAN Models

Generative Adversarial Networks (GANs) are a class of machine learning models that consist of two neural networks, a generator and a discriminator, which are trained simultaneously through a game-theoretic framework. The generator creates data that resembles a target distribution, while the discriminator attempts to distinguish between real data and data generated by the generator.

12.1 GAN Architecture

A GAN consists of two main components: - **Generator G** : Generates synthetic data samples. - **Discriminator D** : Distinguishes between real and generated data samples.

12.1.1 Objective

The objective of a GAN is to train both the generator and the discriminator such that the generator learns to generate data that is indistinguishable from real data.

Let: - x be the real data. - $G(z)$ be the generated data, where z is the input to the generator (typically noise). - $D(x)$ be the probability that x is real, i.e., $D(x) = P(\text{real}|x)$. - $D(G(z))$ be the probability that $G(z)$ is real, i.e., $D(G(z)) = P(\text{real}|G(z))$.

12.1.2 Discriminator Loss

The discriminator tries to maximize the probability of correctly distinguishing real and fake data. The objective function for the discriminator is:

$$\mathcal{L}_D = -\mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] - \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

Where: - $p_{\text{data}}(x)$ is the distribution of real data. - $p_z(z)$ is the distribution of noise (input to the generator).

The first term encourages the discriminator to assign high probability to real data, and the second term encourages the discriminator to assign low probability to fake data generated by the generator.

12.1.3 Generator Loss

The generator tries to fool the discriminator by generating data that is classified as real. The objective function for the generator is:

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z(z)}[\log D(G(z))]$$

The generator minimizes the log of the discriminator's output, meaning it aims to generate data that the discriminator will classify as real.

12.1.4 Minimax Game

The overall GAN training objective is a minimax game between the generator and the discriminator:

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

The generator G tries to minimize this loss, while the discriminator D tries to maximize it.

12.2 Training GANs

The GAN model is trained using the following procedure: 1. **Train the discriminator** D to maximize $\mathcal{L}_D$, i.e., improve its ability to distinguish between real and fake data. 2. **Train the generator** G to minimize $\mathcal{L}_G$, i.e., improve its ability to generate data that the discriminator classifies as real. 3. Repeat the process iteratively until both the generator and the discriminator converge.

12.3 Key GAN Model Architectures

GANs are powerful models for generating synthetic data, with applications in image generation, video generation, and other domains. However, training GANs is challenging, as it requires balancing the generator and discriminator to ensure they both improve simultaneously. Generative Adversarial Networks (GANs) have evolved significantly since their introduction. Various architectures have been proposed to enhance performance in different tasks such as image generation, super-resolution, and style transfer. Below, we review some of the key GAN architectures and their strengths and limitations.

12.4 1. Vanilla GAN

The Vanilla GAN is the original GAN model introduced by Ian Goodfellow and his collaborators in 2014. It consists of a simple architecture with a generator G and a discriminator D , both of which are fully connected neural networks.

12.4.1 Pros:

- Simple and easy to understand.
- Good for basic GAN tasks with simple datasets.
- Serves as the foundation for more advanced GAN architectures.

12.4.2 Cons:

- Prone to training instability.
- Difficulty in capturing complex data distributions.
- Mode collapse: The generator may produce limited variety in outputs.

12.5 2. Deep Convolutional GAN (DCGAN)

DCGAN, introduced by Radford et al. (2015), is a modification of the Vanilla GAN that uses deep convolutional networks in both the generator and the discriminator. It leverages convolutional layers to capture spatial hierarchies in data, particularly for image generation tasks.

12.5.1 Pros:

- Works well for generating high-quality images.

- Stability improvements over the Vanilla GAN due to the use of convolutional layers.
- Better handling of spatial data and images.

12.5.2 Cons:

- Requires large amounts of data to train effectively.
- Still vulnerable to mode collapse in some cases.
- May struggle with high-dimensional data.

12.6 3. Conditional GAN (CGAN)

Conditional GANs, introduced by Mirza and Osindero (2014), condition both the generator and the discriminator on some additional information, such as labels or attributes. This allows the model to generate specific outputs conditioned on input variables.

12.6.1 Pros:

- Enables conditional image generation (e.g., generating images of a specific class).
- Better control over the outputs.
- Can be used for tasks like image-to-image translation.

12.6.2 Cons:

- Requires labeled data for training.
- May suffer from overfitting if not properly regularized.
- Can still experience training instability.

12.7 4. Wasserstein GAN (WGAN)

WGAN, introduced by Arjovsky et al. (2017), modifies the loss function of the GAN to use the Wasserstein distance (also known as Earth-Mover's Distance) instead of the standard Jensen-Shannon divergence. This leads to improved training stability.

12.7.1 Pros:

- Provides a more stable training process compared to Vanilla GANs.
- Easier to train with fewer hyperparameter tuning requirements.
- Can generate high-quality images with less likelihood of mode collapse.

12.7.2 Cons:

- Requires weight clipping or gradient penalty, which can be difficult to implement.
- Slightly slower to train compared to other models due to the optimization of Wasserstein distance.
- Limited application for non-image data tasks.

12.8 5. CycleGAN

CycleGAN, introduced by Zhu et al. (2017), is designed for image-to-image translation tasks without requiring paired training data. It leverages cycle consistency to ensure that an image generated by transforming from domain A to domain B can be transformed back to domain A, preserving its content.

12.8.1 Pros:

- Enables image-to-image translation without paired data.
- Effective for tasks like style transfer and photo enhancement.
- Flexible and widely applicable for unpaired image datasets.

12.8.2 Cons:

- Requires a large amount of training data.
- May produce blurry or low-quality images compared to supervised models.
- Can be computationally expensive and hard to train.

12.9 6. Progressive GAN (ProGAN)

Progressive GANs, introduced by Karras et al. (2018), address the problem of training deep networks by progressively growing both the generator and discriminator. The model starts by training on low-resolution images and gradually increases the resolution during the training process.

12.9.1 Pros:

- Generates high-resolution and highly realistic images.
- Reduces the complexity of training by starting with simpler models.
- Stable training process compared to traditional GANs.

12.9.2 Cons:

- Training can be slow as it requires progressively growing the network.
- Computationally expensive, especially for high-resolution outputs.
- Not ideal for tasks that require real-time or fast inference.

12.10 7. StyleGAN

StyleGAN, introduced by Karras et al. (2019), extends the idea of progressive training by introducing a new architecture where the generator uses a “style” vector to control different levels of detail in the generated images, such as coarse features, textures, and fine details.

12.10.1 Pros:

- State-of-the-art in generating high-quality, realistic images, particularly faces.
- Excellent control over the generated image’s style and attributes.
- Capable of generating extremely high-resolution images.

12.10.2 Cons:

- Extremely computationally expensive to train.
- Requires large datasets for meaningful results.
- Complex architecture, making it harder to implement and understand.

12.11 Summary

Each GAN architecture has its strengths and weaknesses, and the choice of which one to use depends on the specific task and the dataset at hand. While models like DCGAN and WGAN are more stable for general tasks, others like CycleGAN and StyleGAN excel in specific applications such as image translation and high-quality image generation. Careful consideration of the pros and cons is essential to selecting the right model for a given problem.

12.12 Variational Autoencoders (VAEs)

Variational Autoencoders (VAEs) are a class of deep generative models that use variational inference to learn a probabilistic mapping from data to a latent space. A key component of VAEs is the **Kullback-Leibler (KL) Divergence**, which acts as a regularization term to ensure that the learned latent distribution is close to a predefined prior distribution.

In this, we will: - Define KL divergence mathematically. - Use KL divergence to derive the **VAE loss function**. - Explain the **VAE architecture** and how it works.

Chapter 13

2. Kullback-Leibler (KL) Divergence

13.0.1 Definition

KL Divergence measures the difference between two probability distributions. Given two distributions: - $P(x)$ (true distribution) - $Q(x)$ (approximate distribution),

the KL divergence is defined as:

$$D_{KL}(Q(x)||P(x)) = \sum_x Q(x) \log \frac{Q(x)}{P(x)}$$

or in the continuous case:

$$D_{KL}(Q(x)||P(x)) = \int Q(x) \log \frac{Q(x)}{P(x)} dx$$

- If $Q(x)$ is the same as $P(x)$, then $D_{KL} = 0$ (no divergence).
- Otherwise, it quantifies how much $Q(x)$ differs from $P(x)$.
- KL divergence is **not symmetric**, meaning $D_{KL}(Q||P) \neq D_{KL}(P||Q)$.

Chapter 14

3. Derivation of the VAE Loss Function

Variational Autoencoders aim to learn a probabilistic mapping from input data x to a latent space represented by a **latent variable** z . Instead of encoding each x deterministically, VAEs **model the latent space as a probability distribution**.

14.0.1 Bayesian Formulation in VAEs

We define: - $p(x) \rightarrow$ True data distribution (unknown) - $p(z) \rightarrow$ Prior over latent space (typically a normal distribution: $N(0, I)$) - $p(x|z) \rightarrow$ Likelihood (decoder) - $q(z|x) \rightarrow$ Posterior approximation (encoder)

The goal of training a VAE is to maximize the likelihood:

$$\log p(x)$$

Since computing $p(x)$ directly is intractable, we introduce a variational approximation using **ELBO (Evidence Lower Bound)**.

$$\log p(x) = D_{KL}(q(z|x)||p(z|x)) + \mathbb{E}_{q(z|x)}[\log p(x|z)]$$

Rearranging:

$$\log p(x) - D_{KL}(q(z|x)||p(z|x)) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

Since KL divergence is always **non-negative**, the right-hand side forms a lower bound:

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

This **ELBO loss function** balances two terms: 1. **Reconstruction Loss** $\mathbb{E}_{q(z|x)}[\log p(x|z)]$: Ensures generated samples resemble real data. 2. **KL Divergence** $D_{KL}(q(z|x)||p(z))$: Encourages the latent distribution to stay close to the prior (e.g., normal distribution $N(0, I)$).

The final **VAE loss function** is:

$$\mathcal{L} = -\mathbb{E}_{q(z|x)}[\log p(x|z)] + D_{KL}(q(z|x)||p(z))$$

Chapter 15

4. Variational Autoencoder (VAE) Architecture

15.0.1 VAE Structure

A VAE consists of two networks:

1. Encoder (Inference Network)

- Maps input x to latent distribution parameters μ and σ .
- Approximates $q(z|x) = N(\mu(x), \sigma^2(x)I)$.

2. Latent Space Sampling

- Instead of directly sampling from $q(z|x)$, we use the **reparameterization trick**:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim N(0, I)$$

This enables gradient-based optimization.

3. Decoder (Generative Network)

- Maps latent variable z back to data space.
- Models likelihood $p(x|z)$.

15.1 Summary

- KL Divergence ensures that the learned latent space distribution () stays close to a prior distribution $p(z)$.
- VAE optimizes ELBO, balancing reconstruction loss (data fidelity) and KL loss (latent regularization).
- Reparameterization trick enables training via backpropagation.

Key references

1. Kingma, D.P., & Welling, M. (2013). “Auto-Encoding Variational Bayes.”
2. Doersch, C. (2016). “Tutorial on Variational Autoencoders.”

Chapter 16

Why the Field Moved Away from VAEs and GANs

Generative modeling has evolved rapidly over the past decade, with models like Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) playing a major role. However, researchers have increasingly shifted focus to other models due to the limitations of VAEs and GANs. Below, we explore why the field has moved away from these models and what newer approaches have emerged.

16.1 1. Issues with GANs (Generative Adversarial Networks)

16.1.1 a. Training Instability

- **Mode Collapse:** GANs are prone to mode collapse, where the generator produces limited, repetitive outputs. This happens when the generator finds a small subset of outputs that the discriminator incorrectly classifies as real.
- **Vanishing Gradients:** The adversarial loss function used by GANs can lead to vanishing gradients, especially when the discriminator becomes too strong relative to the generator.
- **Difficulty in Convergence:** GANs often struggle with convergence, as both the generator and discriminator need to improve at the same rate, which can be difficult to balance.

16.1.2 b. Lack of Good Evaluation Metrics

- Unlike VAEs, which have a clear likelihood-based objective, GANs lack a standard method to evaluate the quality of the generated samples. Traditional metrics like Inception Score (IS) and Fréchet Inception Distance (FID) are limited in assessing the true diversity and realism of the outputs.

16.1.3 c. Training Challenges

- **Adversarial Nature:** GANs involve a delicate balance between the generator and discriminator. This adversarial setup is sensitive to hyperparameter choices and can lead to training instability, requiring careful tuning to prevent one network from overpowering the other.

16.2 2. Issues with VAEs (Variational Autoencoders)

16.2.1 a. Blurry Outputs

- **Mode Averaging:** VAEs tend to produce blurry outputs because they average over all possible outputs for a given input, as part of the variational inference process. This results in images that lack sharpness and fine details, especially in high-resolution tasks.

16.2.2 b. Poorly Structured Latent Space

- **Latent Space Limitations:** Despite the VAE objective encouraging a regularized latent space, it doesn't always guarantee that the latent variables will be disentangled or meaningfully aligned with the semantics of the data. This leads to suboptimal latent space representations.

16.2.3 c. Inconsistent Trade-Off Between Terms

- The VAE objective function consists of a reconstruction term and a regularization term (KL-divergence), which enforces a prior distribution on the latent variables. Balancing these two terms can be difficult, and improper balancing can result in poor generative performance or overfitting.

16.3 3. Shift Towards New Models

16.3.1 a. Flow-Based Models (Normalizing Flows)

- Flow-based models, like RealNVP and Glow, allow for exact likelihood estimation and have been used to generate high-quality samples. These models invert the generative process, which isn't possible with GANs and VAEs.

16.3.1.1 Pros:

- Exact log-likelihood computation.
- Better sample quality and sharper images than VAEs.

16.3.1.2 Cons:

- High computational cost, both during training and inference.
- Requires reversible transformations, which can be challenging to implement.

16.3.2 b. Autoregressive Models (PixelCNN, PixelSNAIL)

- Autoregressive models generate data one pixel at a time (or one point in sequence) by modeling the joint probability of the data. They have been highly effective for image generation and other structured data tasks.

16.3.2.1 Pros:

- High-quality, sharp images.
- Strong performance in tasks like image denoising and inpainting.

16.3.2.2 Cons:

- Slow inference speed since they generate data sequentially.
- High computational cost during inference.

16.3.3 c. Transformers for Generation

- Transformers, initially used for NLP, have been adapted for image generation tasks (e.g., DALL-E, Vision Transformers). These models excel in scalability and generalization across different data types.

16.3.3.1 Pros:

- State-of-the-art performance on large datasets.
- Flexible for multimodal tasks, including text and image generation.

16.3.3.2 Cons:

- Requires large datasets and computational resources.
- Inference can be slow, especially with large models.

16.3.4 d. Score-based Models (Denoising Diffusion Probabilistic Models)

- Denoising Diffusion Probabilistic Models (DDPM) have gained traction due to their ability to generate high-quality samples with less likelihood of mode collapse compared to GANs.

16.3.4.1 Pros:

- High-quality samples with sharp details and diversity.
- Better robustness to mode collapse.

16.3.4.2 Cons:

- Slow generation process due to multiple diffusion steps.
- Computationally expensive, especially for high-dimensional data.

16.4 4. Why the Shift?**16.4.1 a. Better Evaluation and Stability**

- Newer models like flow-based models and score-based models offer stability and a clear evaluation framework, which is crucial for reliable performance in generative tasks.

16.4.2 b. Improved Sample Quality

- Models like diffusion models provide sharper and higher-quality samples, addressing the blurry outputs of VAEs and the mode collapse issues of GANs.

16.4.3 c. Scalability and Flexibility

- Transformer-based models and flow-based models are highly scalable and flexible, capable of handling diverse data types, including images, text, and even multimodal data. This has made them more versatile for a variety of generative tasks.

16.5 Conclusion

While VAEs and GANs were groundbreaking when introduced, newer generative models such as flow-based models, autoregressive models, and diffusion models offer significant improvements in terms of training stability, sample quality, and scalability. These advancements have led to a shift in focus within the generative modeling community, as they better address the limitations of VAEs and GANs. However, VAEs and GANs still serve as foundational models and remain useful in specific tasks where their strengths are beneficial.

Chapter 17

DIFFUSION MODELS

17.1 1. Introduction to DDPMs

Denoising Diffusion Probabilistic Models (DDPMs) are a class of **generative models** that gradually transform noise into meaningful data through a **reverse diffusion process**. Unlike Variational Autoencoders (VAEs) or Generative Adversarial Networks (GANs), DDPMs model the data distribution by progressively **adding and removing noise** in a **Markovian fashion**.

- **Forward process:** Gradually adds noise to input data x_0 , making it resemble pure Gaussian noise.
- **Reverse process:** Learns to remove noise step-by-step to recover the original data.

This process is governed by KL divergence, leading to a well-defined **variational lower bound (ELBO)** loss function.

17.2 2. Mathematical Formulation of Diffusion Models

Let x_0 be a sample from the data distribution $q(x_0)$. The goal is to model $p_\theta(x_0)$, which generates realistic samples.

17.3 2.1 Forward Diffusion Process

The forward diffusion process **corrupts** the data by adding Gaussian noise at each time step t :

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

where: - β_t is a **variance schedule** that controls noise addition. - x_t is the intermediate noisy image.

By iterating over many steps, x_T approaches a Gaussian distribution:

$$q(x_T) \approx \mathcal{N}(0, I)$$

We can directly sample x_t at any time step using:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

where:

$$\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$$

17.4 3. Reverse Diffusion Process

The goal is to **reverse the forward process** by learning $p_\theta(x_{t-1}|x_t)$:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

- $\mu_\theta(x_t, t)$ is a neural network predicting the denoised sample.
- $\Sigma_\theta(x_t, t)$ represents learned variance.

This step gradually **removes noise**, reconstructing the original image.

17.5 4. Derivation of the DDPM Loss Function Using KL Divergence

To train a DDPM, we **maximize the likelihood** of the data $p_\theta(x_0)$. Since direct computation is intractable, we **optimize the Evidence Lower Bound (ELBO)**:

$$\log p_\theta(x_0) \geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_0 : T)}{q(x_{1:T}|x_0)} \right]$$

Expanding the ELBO:

$$\mathcal{L}_{ELBO} = \mathbb{E}_q \left[D_{KL}(q(x_T|x_0)||p(x_T)) + \sum_{t=1}^T D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)) \right]$$

17.5.1 Key Terms in the Loss Function

1. **Final Step KL Term** $D_{KL}(q(x_T|x_0)||p(x_T))$:

- Since x_T is already close to $\mathcal{N}(0, I)$, this term **vanishes**.

2. **Reconstruction Loss** (Mean Squared Error Formulation):

- Instead of directly minimizing KL divergence, we rewrite the KL term in terms of **noise prediction**:

$$\mathbb{E}_{x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

where:

- $\epsilon \sim \mathcal{N}(0, I)$ is the true noise added to the data.
- $\epsilon_\theta(x_t, t)$ is the predicted noise by the model.

17.5.2 Final DDPM Loss Function

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [w_t \|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

where w_t is an optional weighting factor.

This means DDPMs **learn to predict the noise**, making the model **denoise data at every step**.

To understand the optimizing objective of diffusion models please read:

<https://jehillparikh.medium.com/understanding-diffusion-models-optimization-objective-d7214d36dbd8> Optimizing the Reverse Process Covariance To train DDPMs with a learned reverse process covariance, Σ , the full KL divergence term needs to be optimised⁹. One approach, as described in¹⁰, is to: 1. Train the noise prediction network, ϵ_θ , using L_{simple} . 2. Train Σ using the full KL divergence-based loss function. Generating New Samples Once the DDPM is trained, new images can be generated by starting with random noise, $x_T \sim N(0, I)$, at the final timestep T , and iteratively sampling x_{t-1} from the learned reverse process $p(x_{t-1}|x_t)$ until reaching x_0 . Each sampling step utilises the reparameterisation trick to draw samples from the conditional Gaussian distribution. Advantages of DDPMs High Sample Quality: DDPMs are known for generating high-quality samples, often surpassing the visual fidelity of samples produced by GANs. Stability and Ease of Training: DDPM training is generally more stable than adversarial training used in GANs. Limitations of DDPMs Computational Cost: The iterative nature of both the forward and reverse processes can be computationally expensive, especially for high-resolution images. Slow Sampling: Generating samples using the iterative reverse process can be slow, requiring multiple steps to obtain a final sample.

17.6 IMPLEMENTATION OF DDPM

Forward Process Implementation in Diffusion Models The forward process in diffusion models involves gradually adding Gaussian noise to data samples over a series of timesteps, transforming the data distribution into a simple Gaussian distribution. The implementation of this process is straightforward and involves repeatedly sampling from Gaussian distributions with predefined variances. The forward process is defined as a Markov chain, where each timestep t depends only on the previous timestep $t-1$.

At each timestep, Gaussian noise is added to the sample from the previous timestep. The variance of the added noise is typically controlled by a schedule, which can be linear, cosine, or other forms.

The forward process can be expressed mathematically as:

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I)$$

where:

x_t is the noisy sample at timestep t . α_t is a hyperparameter that determines the variance of the Gaussian noise added at timestep t . It is typically a decreasing function of t , meaning that more noise is added in the early timesteps. I is the identity matrix.

The forward process can be implemented efficiently using the reparameterization trick, which allows sampling from the Gaussian distribution by transforming a sample from a standard normal distribution.

By repeatedly applying the forward process, the data distribution is gradually transformed into a Gaussian distribution with mean 0 and variance 1.

The number of timesteps in the forward process is a hyperparameter that affects the quality of the generated samples and the computational cost of training and sampling

This to sample a noisy image at time T without needing a neural network.

Practical Considerations Noise Schedule: The choice of noise schedule significantly impacts the training and sampling efficiency of diffusion models. Different noise schedules can lead to variations in sample quality and convergence speed.

Number of Timesteps: A larger number of timesteps generally result in a smoother transition to the Gaussian noise distribution but increase the computational cost.

Computational Efficiency: The forward process is computationally efficient as it only involves sampling from Gaussian distributions. However, the computational cost increases linearly with the number of timesteps.

Data Preprocessing: In practice, the input data might be preprocessed before applying the forward process, such as normalisation or resizing, depending on the specific task and dataset.

17.7 Learning the Reverse Process in Diffusion Models

Diffusion models are trained to learn a reverse process that gradually removes the noise added during the forward process, ultimately generating new data samples. This reverse process is modelled as a series of denoising steps, each aiming to recover a slightly less noisy sample from the previous timestep. The success of diffusion models hinges on effectively learning this reverse denoising process. The reverse process is modelled as a Markov chain, similar to the forward process, but operating in reverse time order. The model learns to predict the conditional probability distribution of a less noisy sample, x_{t-1} , given a noisy sample, x_t , from the previous timestep. This conditional probability distribution is typically modelled as a Gaussian distribution: $p(x_{t-1}|x_t) = N(\mu(x_t), \Sigma(x_t))$ where: $\mu(x_t)$ is the predicted mean of the distribution, representing the denoised sample. $\Sigma(x_t)$ is the predicted covariance matrix, capturing the uncertainty in the denoising process. θ represents the model parameters.

The model architecture, often a U-Net or a Diffusion Transformer (DiT), is trained to predict the parameters of this Gaussian distribution. **Training Objective:** The model is trained to minimise the difference between the predicted noise and the ground truth noise added during the forward process. This can be

achieved using various objective functions, including mean squared error (MSE) or variational lower bounds on the data likelihood.

17.8 UNETS

The U-Net architecture consists of an encoder and a decoder pathway connected by skip connections.² The encoder pathway downsamples the input image, extracting features at different resolutions.² The decoder pathway upsamples the encoded features, reconstructing the image at the original resolution.² The skip connections transfer features from the encoder to the decoder at corresponding resolutions, enabling the model to preserve fine-grained details and capture both local and global image information.² In diffusion models, the U-Net takes a noisy image sample, x_t , and its corresponding timestep, t , as input and predicts the noise added to the image or the denoised image, x_{t-1} .² U-Nets for diffusion models often incorporate additional components, such as attention mechanisms and adaptive normalisation layers, to improve performance and enable conditioning on external information.² Attention mechanisms allow the model to selectively focus on specific regions of the image, enhancing its ability to capture complex relationships and generate high-quality samples.² Adaptive normalisation layers, such as Adaptive Layer Normalisation (adaLN), enable the model to condition on external information, such as class labels or text prompts, by modulating the normalisation process based on this information.²³⁴ This allows for controlled generation of samples that adhere to specific constraints or conditions. FOR MODEL DETAILS SEE BLOG POST AND REFERENCE WITHIN

<https://jehillparikh.medium.com/u-nets-with-attention-c8d7e9bf2416>

Chapter 18

1. Introduction to Score-Based Generative Models (SGMs)

Score-Based Generative Models (SGMs), also called **Score Matching with Langevin Dynamics (SMLD)** or **Denoising Score Matching (DSM)**, are a class of diffusion models that **learn the score function** (gradient of the log-likelihood) to generate realistic data.

Unlike **Denoising Diffusion Probabilistic Models (DDPMs)**, which define the process discretely, SGMs generalize diffusion models using **Stochastic Differential Equations (SDEs)** for continuous noise perturbation.

18.0.1 Key Properties

- Forward process transforms data into Gaussian noise.
 - Reverse process learns to **estimate the score function** $\nabla_x \log p_t(x)$.
 - Uses **Score Matching Loss** derived from KL divergence.
-

Chapter 19

2. Mathematical Formulation of Score-Based Models

Let $x_0 \sim p(x)$ be a real data sample. The **goal** is to model the distribution $p(x)$ by learning its **score function** $s_\theta(x_t, t)$.

19.0.1 2.1 Forward Process: Stochastic Differential Equation (SDE)

SGMs define a **continuous diffusion process** via an **SDE**:

$$dx = f(x, t)dt + g(t)dW$$

where: - W is a **Wiener process** (Brownian motion). - $f(x, t)$ is the drift coefficient. - $g(t)$ is the diffusion coefficient. - The noise gradually transforms x_0 into pure Gaussian noise.

Common **SDE choices**: 1. **Variance-Preserving (VP) SDE** (similar to DDPM):

$$dx = -\frac{1}{2}\beta_t x dt + \sqrt{\beta_t} dW$$

2. **Variance-Exploding (VE) SDE**:

$$dx = \sqrt{\frac{d[\sigma^2(t)]}{dt}} dW$$

where $\sigma^2(t)$ controls noise addition.

At large t , x_t becomes Gaussian:

$$q(x_t|x_0) = \mathcal{N}(x_t; x_0 e^{-\frac{1}{2}\beta t}, (1 - e^{-\beta t})I)$$

Chapter 20

3. Reverse Process: Score-Based Generative Model

To **generate data**, we **reverse the SDE**:

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)]dt + g(t)dW$$

Instead of modeling $p_t(x)$, we approximate the **score function**:

$$s_\theta(x_t, t) \approx \nabla_x \log p_t(x)$$

Thus, the reverse process **gradually denoises samples from Gaussian noise**.

Chapter 21

4. Derivation of the Score-Based Loss Function Using KL Divergence

Since direct optimization of likelihood is intractable, we use the **KL divergence between forward and reverse processes**.

$$\mathcal{L} = \mathbb{E}_q \left[D_{KL}(q(x_T|x_0)||p(x_T)) + \int_0^T D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))dt \right]$$

Since $q(x_T) \approx \mathcal{N}(0, I)$, the first term vanishes.

Using **Score Matching**, we rewrite KL as:

$$\mathcal{L}_{score} = \mathbb{E}_{t, x_t} [w(t) \|s_\theta(x_t, t) - \nabla_x \log q(x_t|x_0)\|^2]$$

where $w(t)$ is a weighting function.

21.0.1 Final Score Matching Loss

$$\mathcal{L}_{score} = \mathbb{E}_{t, x_t} \left[\left\| s_\theta(x_t, t) - \frac{x_t - x_0}{\sigma_t^2} \right\|^2 \right]$$

SGMs train by **matching the estimated score function to the true score**.

Score-based diffusion models are another prominent class of diffusion models. They are based on the concept of “scores,” which are the gradients of the log-probability density function of the data distribution. These models learn to estimate the score of the data distribution at different noise levels and then use Langevin dynamics, a sampling method based on stochastic differential equations, to generate new samples. **Score Estimation:** A key component of score-based models is estimating the score function at various noise levels. This is typically done by training a neural network to predict the score of a noisy data sample given its noise level.

Annealed Langevin Dynamics: During sampling, annealed Langevin dynamics is employed to generate samples. This iterative process starts with a sample from a prior distribution and then repeatedly updates the sample by moving it in the direction of the estimated score function, adding noise, and gradually decreasing the noise level over time. Diffusion models IN score based generative models as introduced by Song et al

Score-based generative models, a category that encompasses diffusion models, represent probability distributions through their score functions. The score function is a vector field that points in the direction of the steepest ascent of the log-probability density of the data distribution. This means it indicates the direction in which the likelihood of the data increases most rapidly.

A key advantage of using score functions is that they can be learned from data without requiring adversarial optimisation, unlike models like Generative Adversarial Networks (GANs). This makes training score-based models potentially simpler and more stable.

Score-based models learn the score function by training a neural network, called the score network, to denoise images corrupted with Gaussian noise. This process involves perturbing data with Gaussian noise at multiple scales to enable the network to capture both coarse-grained and fine-grained image features.

Once the score network is trained, new samples can be generated by employing a process called Langevin dynamics. Langevin dynamics is a sampling method that iteratively refines a sample by using the score function to guide it towards regions of higher probability density.

This procedure begins with an initial sample of white noise and gradually denoises it into an image using the learned score network. The iterative updates in Langevin dynamics can be viewed as noisy gradient ascent on the log-density of the data distribution.

A challenge in training score-based models is determining the appropriate noise scales to use during training. Choosing the right noise scales is critical for the success of these models, as using a single noise scale can lead to poor sample quality, particularly for large images.

Intuitively, high noise levels aid in estimating the score function but result in corrupted samples, while low noise levels produce cleaner samples but make

score function estimation more difficult. To balance these trade-offs, score-based models typically leverage multiple noise scales.

Another technique proposed in the source is the use of an exponential moving average (EMA) of model parameters to improve stability during training. This technique has been shown to effectively stabilise training, reduce artifacts in generated samples, and ultimately lead to better FID scores.

Although score-based models have a unified training objective and don't require adversarial training, they can exhibit instability in generated image quality, especially for higher-resolution images. EMA helps mitigate this issue. Here is a detailed explanation of the training and sampling process for score-based models, based on the information provided in the sources: **Training Score-Based Models Objective:** The primary goal of training a score-based model is to learn the score function of the data distribution. The score function, denoted as $\nabla_x \log p(x)$, represents the gradient of the log-probability density function of the data distribution, $p(x)$. This function essentially indicates the direction in which the data distribution's likelihood increases most rapidly.

Score Network: A neural network, called the score network, is trained to approximate this score function. This network takes a data sample, x , as input and outputs a vector that estimates the score at that point.

Denoising Score Matching: A common method for training the score network is denoising score matching (DSM). This approach involves perturbing the training data with Gaussian noise at different scales and training the score network to denoise these corrupted samples.

The training objective in DSM is to minimise the difference between the predicted score by the score network and the true score of the perturbed data distribution. This objective encourages the network to learn how to accurately estimate the score function across various noise levels.

Perturbation with Multiple Noise Scales: Using multiple noise scales during training is crucial for the success of score-based models. High noise levels help estimate the score function at a coarse level, capturing the overall structure of the data distribution, while low noise levels refine the score estimation in the high-density regions, enhancing the fidelity of the generated samples.

Song et al emphasizes the importance of choosing appropriate noise scales and proposes techniques for analytically determining an effective set of noise scales based on the training data. This approach helps to automate the process of noise scale selection, improving training efficiency and model performance.

Noise Conditional Score Network (NCSN): To handle multiple noise scales effectively, a specific architecture called a Noise Conditional Score Network (NCSN) is often employed. This network takes both the data sample, x , and the noise level, σ , as inputs and outputs the estimated score at that point.

The song et al introduces an efficient NCSN parameterization where the score function is computed as $s(x, \sigma) = s(x) / \sigma$, where $s(x)$ is an unconditional score

network. This method simplifies the implementation and can handle many noise scales, even continuous ones, effectively. Sampling from Score-Based Models Langevin Dynamics: Once the score network is trained, it can be used to generate new samples from the learned data distribution. This generation process typically involves a sampling technique called Langevin dynamics.

Langevin dynamics is an iterative process that starts with an initial sample, typically drawn from a simple noise distribution like a Gaussian. The score function, as estimated by the trained score network, is then used to guide the sample towards regions of higher probability density in the data distribution.

Annealed Langevin Dynamics: In practice, a variant called annealed Langevin dynamics is often employed. This approach involves sampling from a series of perturbed data distributions with decreasing noise levels. This gradual decrease in noise levels helps the sampling process converge more effectively to the target data distribution.

Annealed Langevin dynamics as a method that combines information from all noise scales, starting from the largest noise scale and progressively refining the samples by moving to smaller scales.

Iterative Refinement: At each step of Langevin dynamics, the current sample is updated by taking a small step in the direction indicated by the score function, with an added noise term. This noise term introduces stochasticity into the sampling process, helping to explore different regions of the data distribution and preventing the samples from getting stuck in local optima.

As the process progresses, the influence of the noise term diminishes, and the samples gradually converge to the target data distribution, guided by the score function.

Parameter Configuration: The effectiveness of annealed Langevin dynamics heavily relies on properly setting parameters like the number of sampling steps per noise scale, T , and the step size parameter, ϵ .

Song et al provides theoretically motivated guidance for configuring these parameters, aiming to optimise the performance of annealed Langevin dynamics without relying on extensive manual tuning.

Additional Considerations Exponential Moving Average (EMA): The source “2006.09011v2.pdf” also highlights the use of an exponential moving average (EMA) of the score network’s parameters during training. EMA helps to stabilize the training process, especially for high-resolution image generation, where the model might exhibit unstable behaviour despite the unified training objective.

EMA involves maintaining a weighted average of the model’s parameters over the training iterations, with more recent parameters receiving higher weights. This averaging process helps to smooth out fluctuations in the parameter updates, leading to a more stable training trajectory and reducing artifacts in the generated samples.

Chapter 22

Chapter: Reinforcement Learning - Theory and Applications

22.1 1. Introduction

Reinforcement Learning (RL) is a subfield of machine learning that deals with sequential decision-making problems. Unlike supervised learning, where labeled data is used for training, RL agents learn by interacting with the environment through trial and error to maximize a long-term objective, often represented as cumulative rewards.

RL has been widely applied in various domains, including robotics, finance, healthcare, and gaming. The ability of RL to handle complex, dynamic environments makes it a powerful paradigm for artificial intelligence research and real-world applications.

22.2 2. Fundamentals of Reinforcement Learning

22.2.1 2.1 Key Components of RL

1. **Agent:** The entity that makes decisions and interacts with the environment.
2. **Environment:** The external system with which the agent interacts.
3. **State (s):** A representation of the environment at a specific time.

4. **Action ($\mathbf{a}$)**: A decision taken by the agent that affects the environment.
5. **Reward ($\mathbf{r}$)**: A scalar value that provides feedback on the effectiveness of an action.
6. **Policy (π)**: A strategy mapping states to actions.
7. **Value Function ($\mathbf{V}(\mathbf{s})$)**: The expected cumulative reward starting from state s and following policy π .
8. **Q-Function ($\mathbf{Q}(\mathbf{s}, \mathbf{a})$)**: The expected cumulative reward for taking action a in state s and following policy π thereafter.
9. **Discount Factor (γ)**: A factor that determines the importance of future rewards, where $0 \leq \gamma < 1$.

22.2.2 2.2 The RL Process

The RL process can be mathematically formulated as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) , where: - S is the set of possible states. - A is the set of possible actions. - $P(s'/s, a)$ is the transition probability from state s to s' given action a . - $R(s, a)$ is the reward function. - γ is the discount factor.

The goal of RL is to find an optimal policy π^* that maximizes expected cumulative reward.

22.3 3. Types of Reinforcement Learning

22.3.1 3.1 Model-Based vs Model-Free RL

- **Model-Based RL**: The agent builds an internal model of the environment and uses it to make decisions.
- **Model-Free RL**: The agent learns through direct interaction without explicit modeling.

22.3.2 3.2 Value-Based vs Policy-Based RL

- **Value-Based RL**: Focuses on learning the value function (e.g., Q-learning, DQN).
- **Policy-Based RL**: Directly optimizes the policy function (e.g., REINFORCE algorithm).
- **Actor-Critic Methods**: Combines value-based and policy-based approaches (e.g., A2C, PPO).

22.4 4. Popular RL Algorithms

22.4.1 4.1 Q-Learning

Q-learning is a model-free, off-policy algorithm that updates Q-values using the Bellman equation:

$$Q(s, a) = Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate and γ is the discount factor.

22.4.2 4.2 Q-Learning Policy Gradient Theorem

The policy gradient theorem states that the gradient of the expected cumulative reward with respect to policy parameters is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a) \right]$$

This theorem serves as the foundation of policy gradient methods, which optimize policies directly.

22.4.3 4.3 Deep Q-Networks (DQN)

DQN extends Q-learning by utilizing deep neural networks to approximate the Q-function. It stabilizes training through experience replay and a target network.

22.4.4 4.4 Policy Gradient Methods

Policy gradient methods optimize the policy directly using gradient ascent:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a|s) R]$$

Examples: REINFORCE, A2C, PPO

22.5 5. Applications of Reinforcement Learning

- **Robotics:** Training autonomous robots for navigation and manipulation.
- **Gaming:** Mastering complex games like Go (AlphaGo), StarCraft (AlphaStar), and Poker.

- **Finance:** Algorithmic trading, portfolio optimization.
- **Healthcare:** Optimizing treatment strategies and drug discovery.
- **Industrial Automation:** Smart manufacturing and supply chain optimization.

22.6 6. Challenges in RL

- **Exploration vs Exploitation Tradeoff:** Balancing between trying new actions and maximizing known rewards.
- **Sample Inefficiency:** RL models require extensive interaction data.
- **Sparse Rewards:** Some environments provide infrequent feedback.
- **Stability and Convergence Issues:** RL algorithms can be unstable and require extensive tuning.
- **Generalization:** Transferring learned policies to new environments remains challenging.

22.7 7. Conclusion and Future Directions

Reinforcement Learning is a rapidly evolving field with growing applications across industries. Future research aims to address sample inefficiency, stability, and generalization while exploring novel architectures such as meta-learning and self-supervised RL.

For further reading, consider Sutton & Barto's "Reinforcement Learning: An Introduction" and recent advancements in deep RL research papers.

Chapter 23

Markov Decision Process (MDP)

A **Markov Decision Process (MDP)** is a framework for modeling decision-making where outcomes are partly random and partly controlled by an agent. MDPs are widely used in **reinforcement learning (RL)**, **robotics**, **finance**, and **game theory**.

An MDP consists of the following components:

- **States (S):** A set of all possible states the agent can be in.
- **Actions (A):** A set of all possible actions the agent can take.
- **Transition Probability (P):** The probability of moving from one state to another given an action.
- **Reward Function (R):** The reward received for taking an action in a particular state.
- **Policy (π):** A strategy that defines the action selection process in each state.
- **Discount Factor (γ):** A factor that determines how much future rewards are valued compared to immediate rewards.

Mathematically, an MDP is represented as a tuple:

$(S, A, P, R, \pi, \gamma)$

where:

- S is the set of states.
- A is the set of actions.

- $P(s'|s, a)$ is the probability of transitioning from state s to state s' when taking action a .
- $R(s, a)$ is the reward received after taking action a in state s .
- $\pi(a|s)$ is the policy, which defines the probability of taking action a in state s .
- $\gamma \in [0, 1]$ is the discount factor.

23.1 State Transition Probability

The probability of moving from state s to state s' after taking action a is given by:

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$$

This represents the **Markov Property**, meaning the next state **only depends on the current state and action**, not the previous history.

23.2 Policy in MDP

A **policy** (π) defines the agent's behavior by mapping states to actions. Policies can be:

- **Deterministic Policy** ($\pi : S \rightarrow A$): The agent always takes a specific action in a given state.

$$a = \pi(s)$$

- **Stochastic Policy** ($\pi : S \times A \rightarrow [0, 1]$): The agent chooses actions probabilistically.

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s)$$

23.3 Optimal Policy (π^*)

An optimal policy π^* maximizes the expected return:

$$\pi^* = \arg\max_{\pi} V^{\pi}(s), \text{ for all } s \in S$$

where $V^{\pi}(s)$ is the value function under policy π . The optimal policy leads to the highest cumulative rewards over time.

23.4 Bellman Equation

The **Bellman Equation** provides a recursive formula for **value functions**, helping determine the **optimal policy** that maximizes expected cumulative rewards.

23.5 State-Value Function ($V(s)$)

The **value function** of a state s under a policy π is:

$$V^{\pi}(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, \pi \right]$$

Using the **Bellman equation**, we express the recursive relationship:

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} P(s' \mid s, a) \left[R(s, a) + \gamma V^{\pi}(s') \right]$$

where:

- $\pi(a|s)$ is the probability of taking action a in state s .
- $P(s'|s, a)$ is the probability of transitioning to state s' given state s and action a .
- $R(s, a)$ is the immediate reward.
- γ is the discount factor.

23.6 Action-Value Function ($Q(s, a)$)

The **Q-value function** measures the expected reward for taking an action a in state s and then following policy π :

$$Q^{\pi}(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right]$$

The **Bellman equation for Q-values** is:

$$Q^{\pi}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) \sum_{a' \in A} \pi(a' \mid s')$$

23.7 Optimal Policy & Bellman Optimality Equation

The goal of reinforcement learning is to find the **optimal policy** that maximizes the expected cumulative reward. The **Bellman optimality equation** for the optimal state-value function is:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s' \mid s, a) \left[R(s, a) + \gamma V^*(s') \right]$$

Similarly, the **Bellman optimality equation for Q-values** is:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a) \max_{a' \in A} Q^*(s', a')$$

where:

- $V^*(s)$ is the maximum possible value of state s .
- $Q^*(s, a)$ is the maximum expected return for taking action a in state s .

23.8 Conclusion

Markov Decision Processes provide a structured way to **model sequential decision-making problems**, and the **Bellman equation** helps compute optimal strategies. Q-learning is a powerful method for learning action-value functions, while policy learning focuses on directly optimizing the decision-making strategy. These concepts form the foundation of **reinforcement learning algorithms**, including **value iteration**, **policy iteration**, and **deep reinforcement learning**.

23.9 Reinforcement learning

Building on the previous discussion of **Markov Decision Processes (MDP)** and the **Bellman Equation**, we now explore various reinforcement learning

(RL) methods. RL can be broadly categorized into **Model-Based RL** and **Model-Free RL**. These methods help agents learn optimal decision-making strategies within an environment. Additionally, we will briefly cover **Reward-Based Learning**, **Monte Carlo Tree Search (MCTS)**, **Proximal Policy Optimization (PPO)**, and other advanced RL techniques that extend the fundamental concepts of MDPs.

Chapter 24

Reward-Based Learning

Reward-Based Learning is a fundamental concept in reinforcement learning where an agent learns to make decisions by receiving rewards for its actions. The agent's objective is to maximize the total cumulative reward over time. The reward signal serves as feedback to guide the agent toward optimal behavior.

24.1 Types of Rewards

- **Immediate Rewards** ($R(s, a)$): A reward received immediately after taking action a in state s .
- **Delayed Rewards**: A reward that is given after multiple steps, requiring the agent to learn long-term strategies.
- **Sparse Rewards**: Rewards that occur infrequently, making learning more challenging.
- **Dense Rewards**: Frequent rewards that provide continuous feedback.

24.2 Reward Function

The reward function determines the numerical value assigned to each action and state transition. It can be defined as:

```
R: S \times A \to \mathbb{R}
```

where S is the set of states, A is the set of actions, and the function returns a scalar reward value.

24.3 Optimizing Rewards

To maximize cumulative rewards, agents use **return** (G_t), which is the sum of discounted future rewards:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

where γ (discount factor) determines the importance of future rewards.

24.4 Exploration vs. Exploitation

Reward-Based Learning must balance: - **Exploration:** Trying new actions to discover potentially better rewards. - **Exploitation:** Choosing known good actions to maximize immediate rewards.

24.4.1 Methods for Managing Exploration-Exploitation Trade-off:

- **Epsilon-Greedy Strategy:** Randomly selects an action with probability ϵ , otherwise chooses the best-known action.
- **Upper Confidence Bound (UCB):** Prioritizes actions with uncertain rewards.
- **Thompson Sampling:** Uses probability distributions to select actions.

Chapter 25

Model-Based Reinforcement Learning

Model-Based RL refers to methods where an agent **learns a model** of the environment and uses it for decision-making. This typically involves:

1. **Learning the Transition Model** ($P(s'|s, a)$): Predicting the next state given the current state and action.
2. **Learning the Reward Model** ($R(s, a)$): Estimating the reward for taking an action in a given state.
3. **Planning**: Using the learned model to simulate trajectories and improve decision-making.

25.0.1 Popular Model-Based RL Approaches

- **Dynamic Programming (DP)**: Uses Bellman equations to iteratively compute value functions, assuming a known model.
- **Model Predictive Control (MPC)**: Uses a learned model to simulate future states and optimize actions.
- **Dyna-Q**: Combines model-free and model-based learning by integrating planning and learning.

25.1 Monte Carlo Tree Search (MCTS)

MCTS is a search-based decision-making algorithm used in **game AI** and **planning problems**. It simulates future states by exploring the decision tree and backing up estimated rewards, aligning closely with **MDP planning techniques**.

In MCTS, the algorithm uses a model of the environment (specifically, the transition dynamics of states) to simulate future outcomes. During the search process, the algorithm builds a tree of possible future states, which are explored through simulations (rollouts). These simulations rely on the model to predict the outcomes of actions, and the results of the simulations are used to evaluate the best actions to take at each decision point.

The key characteristic of MCTS being model-based is that it requires knowledge of the environment's dynamics to simulate the future. This model could either be explicitly provided (like in a chess game, where the rules of the game are well-defined) or learned through interactions (as in some planning problems for robotics)

25.1.1 MCTS Steps

1. **Selection:** Start from the root and select the most promising node based on a strategy (e.g., Upper Confidence Bound (UCB1)).
2. **Expansion:** Add a new node to the tree when a promising unexplored action is found.
3. **Simulation (Rollout):** Play out random simulations to estimate the value of the new state.
4. **Backpropagation:** Update the values of visited nodes based on the simulation outcome.

25.1.2 Steps of MCTS

1. **Selection:** Starting from the root node, recursively select the child node that maximizes the **Upper Confidence Bound for Trees (UCT)**.

The UCT formula is given by:

$$\text{UCT}(v) = \frac{w(v)}{n(v)} + c \sqrt{\frac{\ln N}{n(v)}}$$

Where:

- $w(v)$ is the total reward for node v .
 - $n(v)$ is the number of times node v has been visited.
 - N is the number of times the parent node has been visited.
 - c is the exploration constant that balances exploration and exploitation.
2. **Expansion:** If the selected node is not a terminal state and has untried actions, add a new child node representing an untried action.

3. **Simulation (Rollout)**: Simulate a random sequence of actions from the current node until a terminal state is reached. The outcome of the simulation gives an estimate of the value of the node.

The return from a simulation is denoted by:

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_k$$

Where:

- G_t is the total reward (return) from time t onwards.
 - γ is the discount factor.
 - R_k is the reward at time step k .
4. **Backpropagation**: After the simulation, backpropagate the results by updating the values and visit counts of all nodes in the path from the current node to the root.

The update rule for a node's value is:

$$\begin{aligned} w(v) &\leftarrow w(v) + G_t \\ n(v) &\leftarrow n(v) + 1 \end{aligned}$$

25.1.3 Applications of MCTS

- AlphaGo (DeepMind)
- Chess and board games
- Robotics planning and decision-making

Chapter 26

Model-Free Reinforcement Learning

Model-Free RL does not explicitly learn a model of the environment. Instead, the agent learns directly from experience by interacting with the environment and optimizing rewards. This method is useful when transition dynamics are unknown or too complex to model.

26.0.1 Two Main Types of Model-Free RL

1. **Value-Based Methods:** Learn a value function and derive a policy from it. **-Q-Learning:** Learns the optimal action-value function without requiring a model. It updates Q-values using the Bellman equation as follows:

$$Q(s, a) = Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- $Q(s, a)$ is the action-value function.
- α is the learning rate.
- $R(s, a)$ is the immediate reward.
- γ is the discount factor.
- $\max_{a'} Q(s', a')$ represents the maximum Q-value for the next state s' .

Q-learning follows an **off-policy** approach, meaning it learns the optimal policy **independent** of the agent's actions.

- **Deep Q-Networks (DQN):** Uses deep neural networks to approximate Q-values, improving scalability.

26.1 Value-Based Learning

Value-Based Learning involves estimating a value function to guide the agent's decision-making process. Instead of learning a policy directly, value-based methods aim to compute the value of states (or state-action pairs) to determine which actions to take.

26.1.1 Value Function

The **value function** $V(s)$ gives an estimate of the expected return from a given state:

$$V(s) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_t | S_0 = s \right]$$

Where: - $V(s)$ is the value of state s , - R_t is the reward at time step t , - γ is the discount factor, and - T is the length of the episode.

26.1.2 Bellman Equation for Value Function

The value function can be computed using the **Bellman equation**, which expresses the value of a state in terms of the values of its successor states:

$$V(s) = \mathbb{E} [R_t + \gamma V(s')]$$

Where s' is the next state after taking an action in state s .

In **Value-Based Learning**, an agent tries to iteratively update the value function until it converges to the optimal value function, which can then be used to select the best action at each state.

26.2 Q-Learning

Q-Learning is a model-free **value-based** reinforcement learning algorithm. It directly learns the **action-value function** $Q(s, a)$, which represents the expected return from taking action a in state s and following the optimal policy thereafter.

26.2.1 Q-Function

The **Q-function** $Q(s, a)$ is defined as:

$$Q(s, a) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_t | S_0 = s, A_0 = a \right]$$

Where: - $Q(s, a)$ is the action-value function for state s and action a , - R_t is the reward at time step t , - γ is the discount factor, - T is the length of the episode.

26.2.2 Bellman Equation for Q-Function

The optimal Q-function can be computed using the **Bellman equation** for $Q(s, a)$:

$$Q^*(s, a) = \mathbb{E} \left[R_t + \gamma \max_{a'} Q^*(s', a') \right]$$

Where: - $Q^*(s, a)$ is the optimal action-value function, - s' is the next state after taking action a , - a' is the next action selected in state s' .

26.2.3 Q-Learning Update Rule

Q-Learning updates the Q-values iteratively based on the observed rewards and next state-action pairs. The update rule is as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Where: - α is the learning rate, - γ is the discount factor, - R_{t+1} is the reward received after taking action a_t in state s_t , - $\max_{a'} Q(s_{t+1}, a')$ is the maximum Q-value for the next state s_{t+1} .

26.2.4 Exploration vs. Exploitation

Q-Learning uses an **exploration vs. exploitation** strategy where the agent explores random actions (exploration) and selects actions with the highest Q-value (exploitation). This is typically achieved using an **ϵ -greedy strategy**:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg \max_a Q(s_t, a) & \text{with probability } 1 - \epsilon \end{cases}$$

Where: - ϵ is the exploration rate, which is usually decayed over time.

26.2.5 Benefits of Q-Learning

- **Model-Free:** Q-Learning does not require a model of the environment and can be used in any Markov decision process (MDP).
 - **Optimal Policy:** Q-Learning converges to the optimal policy if given enough time and proper exploration.
-

26.3 Deep Q-Network (DQN)

Deep Q-Networks (DQN) extend the **Q-learning** algorithm by approximating the Q-function using a deep neural network. This is useful for environments with high-dimensional state spaces where tabular Q-learning is not feasible.

26.3.1 DQN Overview

Instead of maintaining a Q-table, DQN uses a **deep neural network** $Q(s, a; \theta)$ to approximate the Q-values for state-action pairs, where: - s is the state, - a is the action, - θ are the parameters of the neural network (weights).

26.3.2 DQN Update Rule

The update rule for DQN is similar to the Q-learning update rule, but it uses the neural network to approximate the Q-values:

$$Q(s_t, a_t; \theta) \leftarrow Q(s_t, a_t; \theta) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta) \right]$$

Where: - $Q(s_t, a_t; \theta)$ is the Q-value predicted by the neural network at time step t , - θ^- is the target network's parameters, which are updated periodically to stabilize learning.

26.3.3 Experience Replay

To break correlations between consecutive samples and improve sample efficiency, DQN uses an **experience replay** buffer. The agent stores its experiences in a buffer and samples mini-batches to update the Q-network, which helps reduce the correlation between consecutive updates and improves convergence.

The experience replay process involves: - Storing the agent's experiences $(s_t, a_t, R_{t+1}, s_{t+1})$ in the replay buffer. - Sampling a random mini-batch from the buffer to compute the Q-value updates.

26.3.4 Target Network

To stabilize training, DQN uses a **target network** to generate stable Q-value targets. The target network is a copy of the Q-network, but its weights are updated less frequently (e.g., every few thousand steps). This helps avoid instability caused by frequent updates to both the policy and the Q-values.

26.4 Applications of DQN

DQN has been successfully applied to a variety of complex problems, including:

- **Atari Games:** DeepMind demonstrated DQN's success in learning to play Atari games directly from pixel input, surpassing human-level performance in many games.
- **Robotics:** DQN has been applied to robot control tasks, enabling robots to learn optimal behaviors in environments with high-dimensional sensory inputs.
- **Autonomous Vehicles:** DQN can be used for decision-making in autonomous driving, where the state space can be high-dimensional due to the complex environment.
- **Healthcare:** DQN can optimize treatment plans by learning policies for patient care, taking into account complex and high-dimensional data from medical histories.

2. **Policy-Based Methods:** Learn a direct mapping from states to actions.

Policy learning focuses on directly learning a **policy function** $\pi(a|s)$ without explicitly estimating the value function.

26.4.1 Types of Policy Learning

1. **Deterministic Policy Learning:**

$$a = \pi(s)$$

The agent always takes a specific action in a given state.

2. **Stochastic Policy Learning:**

$$\pi(a | s) = \Pr(A_t = a | S_t = s)$$

The agent selects an action based on a probability distribution.

26.4.2 Policy Gradient Methods

Policy gradient methods optimize the policy directly by maximizing the expected cumulative reward:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t R_t \right]$$

where θ represents the policy parameters. The policy is updated using gradient ascent:

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

26.5 Policy Gradient Methods

In **Policy Gradient Methods**, the policy is typically represented as a probability distribution over actions given a state:

$$\pi(a|s)$$

The goal is to optimize the expected return by adjusting the policy parameters θ to increase the likelihood of actions that lead to higher rewards. The **optimization objective** for policy gradient methods is often written as:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t R_t \right]$$

where: - π_{θ} is the current policy parameterized by θ , - R_t is the return (reward) at time step t , - γ is the discount factor, - T is the length of the episode.

26.5.1 Policy Update

Policy gradient methods update the policy parameters θ using the gradient of this objective function:

$$\theta = \theta + \alpha \nabla_{\theta} J(\theta)$$

where α is the learning rate.

26.5.2 Policy Gradient Theorem

The gradient of the expected return can be derived as:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) A_t]$$

where A_t is the **advantage function**, which measures how much better an action is compared to the average action taken in the state.

26.6 Types of Policy Gradient Methods

26.6.1 REINFORCE

- A Monte Carlo method that computes the return over an entire episode and uses it to update the policy.

26.6.2 Actor-Critic Methods

- Combines the policy gradient (actor) and value function (critic) to reduce variance in the policy updates.

26.6.3 Proximal Policy Optimization (PPO)

- A more advanced method that uses a clipped objective to ensure stable updates.

26.7 REINFORCE (Monte Carlo Policy Gradient): Uses episode returns to update policies.

REINFORCE is a **Monte Carlo Policy Gradient** method used for learning a policy in **Reinforcement Learning** (RL). The key idea behind REINFORCE is to directly optimize the policy by using the **gradients** of the expected return with respect to the policy parameters. The algorithm is based on **Monte Carlo** methods, meaning it requires complete episodes to estimate the policy gradients.

The main goal of **REINFORCE** is to optimize the **policy function** $\pi(a|s, \theta)$, where θ represents the parameters of the policy (e.g., neural network weights).

The **objective** in policy gradient methods is to maximize the **expected return** (reward), which can be represented as:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R_t \right]$$

Where: - $J(\theta)$ is the objective function to maximize. - R_t is the reward received at time step t . - γ is the discount factor. - T is the total number of time steps in an episode. - π_θ is the policy parameterized by θ .

26.7.1 Gradient of the Objective

To optimize the policy, we take the gradient of $J(\theta)$ with respect to the parameters θ :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R_t]$$

This gradient tells us how to adjust the parameters θ to maximize the expected return.

26.8 REINFORCE Algorithm Steps

1. **Generate an Episode:** Follow the current policy $\pi_\theta(a|s)$ to generate an episode, which consists of states s_t , actions a_t , and rewards R_t .
2. **Compute the Return:** For each time step t , compute the return G_t from the current time step to the end of the episode. The return is the total accumulated reward:

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_k$$

3. **Update the Policy:** The policy parameters are updated using the following rule:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

Where:

- α is the learning rate.
- $\nabla_\theta \log \pi_\theta(a_t|s_t)$ is the policy gradient for the action a_t taken in state s_t .
- G_t is the return calculated in step 2.

26.9 Proximal Policy Optimization (PPO)

PPO is a robust reinforcement learning algorithm that improves upon basic policy gradient methods like REINFORCE by introducing a clipped objective to stabilize training and improve sample efficiency. Its main advantage lies in its ability to limit large policy updates, which helps to maintain the stability of the learning process, especially in complex environments.

PPO has become one of the most popular reinforcement learning algorithms due to its efficiency, ease of implementation, and strong empirical performance across a variety of tasks.

The core idea in PPO is to optimize the policy by using a **clipped objective function**. The goal is to maximize the **expected return** while ensuring that updates to the policy are not too large.

The PPO objective is:

$$L(\theta) = \mathbb{E} [\min (r_t(\theta)A_t, \text{clip} (r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

Where: - $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the current policy and the old policy. - A_t is the advantage function, which estimates how much better the action a_t is compared to the average action in state s_t . - ϵ is a small clipping parameter (typically $0.1 \leq \epsilon \leq 0.3$). - The **clip** function ensures that the policy doesn't change drastically between updates, which helps in stabilizing training.

26.9.1 Why Clipping Helps:

The **clipping** mechanism in the objective function prevents the probability ratio $r_t(\theta)$ from becoming too large. When $r_t(\theta)$ exceeds the threshold $1 + \epsilon$ or $1 - \epsilon$, the update is limited to avoid overly large changes in the policy, ensuring a more **stable training process**.

26.10 The Advantage Function

The **Advantage Function** A_t is used to indicate how much better an action is compared to the baseline (i.e., the expected action-value). The advantage can be computed as:

$$A_t = \hat{Q}(s_t, a_t) - V(s_t)$$

Where: - $\hat{Q}(s_t, a_t)$ is the action-value function, representing the total expected reward following action a_t at state s_t . - $V(s_t)$ is the value function, representing the expected return from state s_t .

The advantage function helps to reduce the **variance** of the updates and stabilizes training by ensuring that actions that lead to better-than-expected outcomes receive higher rewards.

26.11 PPO Algorithm Steps

1. **Generate Trajectories:** Collect a batch of episodes by interacting with the environment using the current policy π_θ .
2. **Compute Advantages:** For each trajectory, compute the advantage function A_t to estimate how good each action was relative to the baseline.
3. **Optimize the Objective:** Update the policy parameters θ by maximizing the clipped objective function $L(\theta)$.
4. **Repeat:** After each policy update, continue collecting new trajectories and update the policy based on the new data.

26.11.1 Key Features of PPO

- Uses **trust region updates** to avoid large policy changes, maintaining stability.
- Balances exploration and exploitation efficiently.
- More sample-efficient compared to older methods like REINFORCE.

26.12 Comparison with REINFORCE

Aspect	REINFORCE	PPO
Variance	High variance. REINFORCE estimates the policy gradient from a single trajectory, which results in high variance and inefficient learning.	Lower variance. PPO uses the advantage function and clipped objective to reduce variance in policy updates.
Clipping	No clipping mechanism. There is no mechanism to limit large policy updates, which can lead to instability in training.	Clipping mechanism. PPO limits the magnitude of policy updates, preventing drastic changes in the policy, which leads to more stable and reliable training.

Aspect	REINFORCE	PPO
Policy Update Mechanism	Simple policy gradient. REINFORCE updates the policy based on a Monte Carlo return, without incorporating any mechanism to control the magnitude of the policy update.	More advanced. PPO incorporates a clipped objective, making use of advantage estimation to better guide the policy updates.
Sample Efficiency	Low sample efficiency. REINFORCE tends to be inefficient in making good use of the collected data.	High sample efficiency. PPO utilizes multiple trajectories and makes more efficient use of collected data.

26.13 Applications of PPO

PPO is a widely used algorithm in reinforcement learning due to its stability and sample efficiency. It is applied in various domains, including:

- **Robotics:** Learning control policies for robots to perform tasks such as manipulation, walking, and navigation.
- **Game Playing:** Used in training AI for complex games (e.g., **Dota 2**, **Atari games**, and **MuJoCo environments**).
- **Natural Language Processing:** PPO has been used in sequence generation tasks like text summarization and dialogue generation.
- **Autonomous Vehicles:** In self-driving cars, PPO can be used for decision-making and planning in complex environments.

26.14 Actor-Critic Methods

Actor-Critic methods combine both **policy gradient** (actor) and **value function** (critic) to improve the efficiency of learning. The idea is to have two separate networks:

- **Actor:** The policy function $\pi(a|s)$, which directly determines the action taken by the agent.
- **Critic:** The value function $V(s)$ or **advantage function** $A(s, a)$, which estimates the value of states or the advantage of actions.

26.14.1 Objective Function

The objective for Actor-Critic methods is to update the policy based on the **advantage function**, which helps reduce variance in the policy updates.

The total objective function for **Actor-Critic** methods is:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t A_t \right]$$

Where: - π_{θ} is the current policy parameterized by θ , - A_t is the advantage at time step t .

26.14.2 Advantage Function

The **advantage function** is defined as:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

where: - $Q(s_t, a_t)$ is the action-value function, representing the expected return from taking action a_t in state s_t , - $V(s_t)$ is the value function, representing the expected return from state s_t .

The **advantage function** $A(s_t, a_t)$ tells us whether an action is better or worse than the average action in that state.

26.14.3 Policy Update (Actor)

The **actor** updates the policy by following the gradient of the expected return with respect to the policy parameters:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t]$$

26.14.4 Value Update (Critic)

The **critic** updates the value function by minimizing the **temporal difference (TD) error**:

$$\delta_t = R_t + \gamma V(s_{t+1}) - V(s_t)$$

Where: - δ_t is the TD error at time step t , - R_t is the reward at time step t , - $V(s_{t+1})$ is the estimated value of the next state.

The critic is updated by minimizing the loss:

$$L(\theta_{\text{critic}}) = \mathbb{E}[(\delta_t)^2]$$

26.14.5 Benefits of Actor-Critic Methods

- **Low Variance:** By using the critic to estimate the value, Actor-Critic methods reduce the variance in the policy updates.
 - **Efficient Learning:** The use of the advantage function helps in better exploration and exploitation.
 - **On-Policy:** Typically, Actor-Critic methods are on-policy, meaning the agent learns directly from the actions taken by the current policy.
-

26.15 Types of Actor-Critic Methods

- **Deep Deterministic Policy Gradient (DDPG):** An off-policy actor-critic method that is used for continuous action spaces.
- **Asynchronous Advantage Actor-Critic (A3C):** A multi-agent version of the Actor-Critic method that runs multiple agents in parallel to speed up training.
- **Trust Region Policy Optimization (TRPO):** A more advanced version of Actor-Critic that uses trust regions to ensure stable updates.

Chapter 27

Conclusion

Reinforcement Learning builds upon **Markov Decision Processes (MDP)** by developing model-based and model-free approaches. Reward-based learning is the core principle behind RL, guiding the agent to optimize decision-making. Methods such as **MCTS** and **PPO** have been instrumental in advancing AI applications in robotics, gaming, and autonomous control. Understanding these methods in relation to MDP concepts enables more efficient and scalable RL solutions.

Chapter 28

Introduction

Image encoders have evolved significantly with the emergence of **self-supervised learning** techniques. Recent advancements like **DINO** and **DINOv2** have demonstrated strong performance in learning feature representations without labeled data. In this post, we explore self-attention, transformers, and how they lead to advanced vision encoders.

Chapter 29

Self-Attention Mechanism

Self-attention is a key component of transformer architectures and is responsible for capturing **long-range dependencies** in input sequences. It enables the model to assign different importance weights to various parts of the input, allowing for more dynamic feature extraction.

29.1 Mathematical Formulation

Given an input sequence of token embeddings X , we compute **Query (Q)**, **Key (K)**, and **Value (V)** matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where W_Q , W_K , and W_V are learnable weight matrices. These matrices help in computing attention scores and refining feature representations.

The **scaled dot-product attention** is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where d_k is the dimension of the key vectors, which stabilizes gradient updates.

29.2 Multi-Head Attention

Instead of computing a single attention function, transformers use **multi-head attention**, which allows the model to focus on different parts of the sequence simultaneously. The attention heads are computed as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W_O$$

where each attention head computes attention independently before being concatenated and linearly transformed.

Chapter 30

Transformer Encoder Architecture

The **Transformer encoder** consists of multiple stacked layers, each containing:

- Multi-head self-attention mechanism
- Feed-forward network (FFN)
- Residual connections and Layer Normalization

Each encoder block follows this sequence:

1. Input embeddings + Positional Encoding
2. Multi-head self-attention
3. Layer Normalization
4. Feed-forward network
5. Layer Normalization (again)

The encoder processes the input in parallel, unlike RNNs, making it more efficient for long sequences.

30.1 Masked Language Modeling (MLM)

MLM is commonly used in natural language processing (NLP) tasks, where a fraction of the input tokens are randomly masked, and the model learns to predict the missing tokens based on their context. The objective function for MLM is:

$$\mathcal{L}_{MLM} = - \sum_{i \in M} \log P(x_i | x_{\setminus i})$$

where M represents the set of masked positions, and $P(x_i | x_{\setminus i})$ is the probability of correctly predicting the masked token.

Chapter 31

Vision Transformers (ViTs)

ViTs apply **self-attention** to image data, treating images as sequences of patches rather than using convolutional operations.

31.1 Patch Extraction and Encoding

An image is split into patches, flattened, and passed through a **linear projection**:

$$X_{patch} = W_E \cdot X_{input} + b_E$$

where W_E is the embedding matrix and b_E is a bias term. These embeddings are then processed using self-attention layers to extract contextual information.

Chapter 32

Contrastive Language-Image Pretraining (CLIP)

CLIP is a multimodal self-supervised learning model designed to learn joint representations of images and text. It is trained on a large dataset of image-text pairs using a contrastive learning approach.

32.1 Architecture

CLIP consists of two main components:

- An image encoder (ResNet or Vision Transformer)
- A text encoder (Transformer-based model)

Both encoders project images and text into a shared embedding space, where semantically similar images and text have higher similarity scores.

32.2 Training Objective

CLIP uses a contrastive learning objective where it maximizes the cosine similarity between matching image-text pairs while minimizing similarity between mismatched pairs:

$$\mathcal{L}_{CLIP} = - \sum_i \log \frac{e^{\cos(I_i, T_i)/\tau}}{\sum_j e^{\cos(I_i, T_j)/\tau}}$$

where I_i and T_i are corresponding image-text pairs, and τ is a temperature parameter that controls the sharpness of the distribution.

32.3 Applications of CLIP

- Zero-shot image classification
- Image retrieval and search
- Generating text-based descriptions of images
- Few-shot learning for downstream vision tasks

CLIP's ability to understand images and text jointly makes it a powerful model for many vision-language applications.

Chapter 33

DINO: Self-Supervised Vision Encoding

DINO (Self-Distillation without Labels) is a **self-supervised learning method** that trains a **student** network to predict the output of a momentum **teacher** network.

33.1 Key Features of DINO

- Works with both CNNs and ViTs
- Learns representations using **contrastive learning**
- Uses a momentum teacher-student setup
- Multi-crop augmentation for efficient feature learning
- Achieves competitive performance in unsupervised representation learning

33.2 DINO Training Process

DINO minimizes the cross-entropy loss between teacher and student outputs:

$$\mathcal{L}_{DINO} = - \sum_i p_i^T \log p_i^S$$

where p_i^T and p_i^S are teacher and student predictions. The teacher network updates using an exponential moving average of the student's weights.

Chapter 34

DINOv2: Improved Vision Encoding

DINOv2 extends DINO with **Masked Image Modeling (MIM)** and **Self-Supervised Instance Discrimination**, improving its ability to capture both local and global features.

34.1 Key Improvements in DINOv2

- Introduces **masked image modeling (MIM)** to learn local textures
- Uses **multi-crop augmentation** to enhance representation learning
- No reliance on **text supervision** (unlike CLIP)
- Improved transferability to **dense prediction tasks** (e.g., segmentation)
- Asymmetric teacher-student design for more effective training
- Incorporates KoLeo regularization to improve feature uniformity

34.2 Masked Image Modeling (MIM)

MIM extends the concept of MLM to images by randomly masking patches in an image and training the model to reconstruct the missing information. The objective function for MIM is:

$$\mathcal{L}_{MIM} = - \sum_{i \in M} \log P(I_i | I_{\setminus i})$$

where I_i represents the masked image patch, and $I_{\setminus i}$ denotes the observed patches.

34.3 Training Objectives

DINOv2 uses a **hybrid objective**:

$$\mathcal{L}_{DINOv2} = \lambda_1 \mathcal{L}_{MIM} + \lambda_2 \mathcal{L}_{Instance}$$

where:

- $\mathcal{L}_{MIM}$ is the loss for **masked image modeling**, focusing on reconstructing masked patches
- $\mathcal{L}_{Instance}$ is the loss for **instance discrimination**, ensuring robust feature alignment across augmentations

DINOv2 leverages a **vision-only training approach**, making it highly effective for self-supervised learning in image-based tasks.

Chapter 35

Conclusion

DINO and DINOv2 represent significant advances in **self-supervised learning for vision tasks**. Their ability to learn rich representations **without labeled data** makes them valuable for diverse applications such as **image classification, retrieval, segmentation, and biomedical imaging**.